

## Output Variables

| Parameter Name             | Mapped Context Tag     | Context Tag's Description                     |
|----------------------------|------------------------|-----------------------------------------------|
| Rate Adjustment By Tier ID | RateAdjustmentByTierID | The ID of the rate adjustment by tier record. |

## Configure Your Products in Revenue Cloud

Use the Product Configurator tool to customize the components and attributes of a product so that the final configured product meets the business requirement expectations. Product configuration entails customizing a product to align with the specific requirements of a customer.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

By using the Product Configurator, present complex product offerings to customers efficiently. The Product Configurator tool provides users with an optimized and intuitive interface making it simple and easy to select the most suitable combination of features for a complex product. The Product Configurator not only facilitates the configuration of product bundles, but it also facilitates the customization of individual product attributes.

With Product Configurator, your sales representatives can validate product selections instantly and get immediate cost insights with real-time pricing updates. The Product Configurator proves to be a valuable asset for businesses that offer complex products. It contributes to saving time and resources, enhancing accuracy, and elevating the customer experience.

Some of the key features of the Product Configurator tool are:

- Customize the product configurator flow: The product configurator flow with purpose-built user interface components offers a ready-made foundation for your product configurator. Customize and extend the Product Configurator flow to create a unique look and feel that aligns with your brand.
- Preview runtime experience: Assign multiple products or product classes to a product configurator flow and preview the user experience in real-time.
- Build complex product bundles with ease: Product Configurator's intuitive interface and headless APIs support multi-level product bundle structures. Users can effortlessly configure quantities, attributes, and options while receiving real-time pricing updates.
- Customizable configuration rules: Manage the attribute structure in the product bundle with cardinality check, product selection validations, and messages.

[Learn and Explore Product Configurator](#)

Learn how your users can use the product configurator. Understand its core features to provide a smooth and user-friendly buying experience for your customers.

### **Enable Product Configurator**

Grant users access to use the Product Configurator features and configure products at run time.

### **Product Configurator Setup**

Product Configurator provides a framework to define and apply rules and constraints when configuring products. It features two rule engines: Business Rules Engine and Constraint Rules Engine. Use Configurator with Business Rules Engine for simple product configuration rules, and opt for Configurator with Constraint Rules Engine for handling large and complex product configuration rules and constraints. If you enable both the rule engines, you can specify which rule engine to use by default. The setup steps help to get started with using Product Configurator.

### **Explore the Product Configurator Flow**

The product configurator flow outlines the layout for presenting customizable product information in the user interface.

### **Work With the Product Configurator**

Create complex product configurations with ease by using Product Configurator. Define attributes, variants, and metadata. Test and verify your product configurations with the built-in runtime experience. Your users can use the product configurator to easily configure the product options during the purchase process and view the relevant pricing.

### **Business Rules Engine and Constraint Rules Engine**

Product Configurator provides the Business Rules Engine and the Constraint Rules Engine. To work with simple product configuration rules, use the Business Rules Engine. To work with large and complex product configuration rules, use the Constraint Rules Engine. If you enable both the rules engines, you can specify which rules engine to use by default, and you can also enable sales reps to choose which rules engine to use for each transaction.

### **Rules in Configurator With Business Rules Engine**

When you configure products for sale, ensure that all your products have valid attributes and are compatible with other products in the offering. With configuration rules in Product Catalog Management, you can ensure that your users select the right set of products and that the product configuration aligns with the business needs and standards.

### **Rules and Constraints in Configurator With Constraint Rules Engine**

To create configuration rules and constraints that can validate large and complex product configurations, use Constraint Rules Engine.

### **Third-Party Configurator**

Revenue Cloud provides extensibility options to third-party users seeking to build integrations with the revenue cloud product. Revenue Cloud supports various third-party use cases. The support includes replacing the first-party configurator with the third-party configurator, integrating third-party UI components into the first-party configurator, and creating custom configurator UI using headless configurator APIs.

## Learn and Explore Product Configurator

Learn how your users can use the product configurator. Understand its core features to provide a smooth

and user-friendly buying experience for your customers.

## REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

### Product Configurator Key Terms

Let's look at some key terms that you can come across while using Product Configurator.

#### See Also

[Explore the Product Configurator Flow](#)

[Work With the Product Configurator](#)

## Product Configurator Key Terms

Let's look at some key terms that you can come across while using Product Configurator.

## REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

Let's look at some key terms that you can come across while using Product Configurator.

| Term                              | Description                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Summary Component                 | This component displays a summary of the configured product bundle, including prices and attributes. As the user makes choices during product configuration, the updates are reflected in this component. Additionally, the component provides a detailed breakdown of the prices for the entire configuration. Selections are categorized according to their corresponding products, clearly illustrating the overall bundle structure. |
| Option Groups                     | This component shows child products of the bundled product in a specified format.                                                                                                                                                                                                                                                                                                                                                        |
| Message Component                 | This component shows messages for warnings and errors that occur during configuration. The messages convey information, an error, an alert, or a warning.                                                                                                                                                                                                                                                                                |
| Default Product Configurator Flow | Sample flow that is shipped with the product. Users can use this flow as a template to create a customized product configuration flow as per their business requirements.                                                                                                                                                                                                                                                                |

| Term               | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Instant Pricing    | Instant pricing provides the users an option to get the latest pricing. On enabling instant pricing, any changes made to product selections are reflected immediately in the prices shown in the summary component.                                                                                                                                                                                                                                                                                                                                      |
| Product Validation | This capability enables users to explicitly trigger qualification, Product Catalog Management validation, and configuration rules only when necessary, rather than having these rules run automatically with every product selection. This approach enhances performance and allows users to address potential configuration errors before saving. Since users are charged for executing configuration rules, this setting encourages them to use these rules judiciously and only when needed, ultimately improving the overall performance experience. |
| Rule Scope         | A configuration rule executes within the boundary of the rule scope. The rule scope can be Bundle or Product or Transaction. When the rule scope is Bundle, the configuration rule executes within the boundaries of the root product of the bundle. When the rule scope is Product, the configuration rule executes for the product irrespective of where the product is in the quote or order. When the rule scope is Transaction, the configuration rule executes at the transaction level (quote or order) beyond the existing bundle or product.    |

## Enable Product Configurator

Grant users access to use the Product Configurator features and configure products at run time.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

### USER PERMISSIONS NEEDED

---

|                                     |                      |
|-------------------------------------|----------------------|
| To enable the product configurator: | Product Configurator |
|-------------------------------------|----------------------|

---

1. In Setup, in the Quick Find box, enter *Revenue Settings*, and then click **Revenue Settings**.
2. Turn on Configure Products at Runtime.

### Product Configurator Users, Profiles, and Permission Sets

You can manage Product Configurator functionality easily and securely by using permission set licenses. Permissions are added for specific user personas to complete the work that's assigned to their role.

### User Permissions for Product Configurator

User permissions specify which tasks users can perform and which features users can access.

## Product Configurator Users, Profiles, and Permission Sets

You can manage Product Configurator functionality easily and securely by using permission set licenses. Permissions are added for specific user personas to complete the work that's assigned to their role.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

## Create Users and Profiles

To begin, create users for Product Configurator. Then assign users the appropriate permission sets. To help you plan, refer to the permission set table for Product Configurator.

When you create a user, you must also assign a profile. Profiles define default settings for users. Some organizations create their own profiles, while others choose to use profiles included with Salesforce.

Remember, users can have only one profile, but can have many permission sets assigned to them.

## Assign Permissions

Permission Set License

| Permission Set License Name | Description                                                                       |
|-----------------------------|-----------------------------------------------------------------------------------|
| Product Configuration User  | Provides create, read, update, and delete access to Product Configurator objects. |

Permission Set

| Permission Set Name                        | Description                                                                                |
|--------------------------------------------|--------------------------------------------------------------------------------------------|
| Product Configurator                       | Provides users read-write access to Product Configurator objects and APIs.                 |
| Product Configuration Rules Designer       | Create and manage product configuration rules in Configurator with Business Rules Engine.  |
| Product Configuration Constraints Designer | Create and manage constraint types and rules in Configurator with Constraint Rules Engine. |

### See Also

[View and Manage Users](#)

[Create or Clone Profiles](#)

[View and Manage Your Permission Set Licenses](#)

## User Permissions for Product Configurator

User permissions specify which tasks users can perform and which features users can access.

### REQUIRED EDITIONS

Available in: Lightning Experience

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

| User Permissions                                 |                                                                                                     |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| User Permission Name                             | Description                                                                                         |
| Product Configurator API User                    | Provides users read-write access to Headless Configurator APIs.                                     |
| Product Configurator                             | Provides users read-write access to Product Configurator objects.                                   |
| Manage Configurator with Business Rules Engine   | Create, update, and delete configuration rules in Configurator with Business Rules Engine.          |
| Manage Configurator with Constraint Rules Engine | Create, update, and delete constraint types and rules in Configurator with Constraint Rules Engine. |

## View and Assign Permission Set Licenses in Product Configurator

Understand permission set licenses in Product Configurator and how to assign them to users.

### REQUIRED EDITIONS

Available in: Lightning Experience

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

| USER PERMISSIONS NEEDED  |                      |
|--------------------------|----------------------|
| To view permission sets: | Product Configurator |

Permission set licenses extend the functionality of a Salesforce user license and provide access to features and objects. Some permission set licenses are associated with a permission set, and when you assign the permission set to a user, you automatically assign the permission set license. Other permission

set licenses are assigned directly to a user. Users need both the appropriate permission sets and permission set licenses to work in Product Configurator.

If you have questions about the permission set licenses in your org, contact your Salesforce account executive.

1. From Setup, in the Quick Find box, search for and then select **Company Information**.
2. Select the Permission Set Licenses related list and, if necessary, click **Go to list** to see the complete list.
3. Click any license name to see its details and assign it to users.

## Product Configurator Setup

Product Configurator provides a framework to define and apply rules and constraints when configuring products. It features two rule engines: Business Rules Engine and Constraint Rules Engine. Use Configurator with Business Rules Engine for simple product configuration rules, and opt for Configurator with Constraint Rules Engine for handling large and complex product configuration rules and constraints. If you enable both the rule engines, you can specify which rule engine to use by default. The setup steps help to get started with using Product Configurator.

### REQUIRED EDITIONS



**Note** Constraint Rules Engine services aren't available in Government Cloud and in the orgs within the [EU Operating Zone \(OZ\)](#). For more information, contact your Salesforce account executive.

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

#### [Set Up Configurator With Business Rules Engine](#)

Define configuration rules using the Business Rules Engine and invoke the rules to validate product configurations.

#### [Set Up Configurator With Constraint Rules Engine](#)

Set up the Constraint Rules Engine to validate large and complex product configurations. Enable the Constraint Rules Engine and create the ConstraintEngineNodeStatus custom field on transaction line item objects. Map this custom field to order, quote, and asset mappings within the active context definition.

#### [Specify Which Rules Engine to Use](#)

Use the Transaction Type field in a quote or order to specify the rule engine to use for validating product configurations and executing configuration rules and constraints. Create Transaction Processing Type records based on your requirements, and then specify the default rule engine on the Revenue Settings page to validate product configurations and execute rules and constraints.

## Set Up Configurator With Business Rules Engine

Define configuration rules using the Business Rules Engine and invoke the rules to validate product configurations.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

### USER PERMISSIONS NEEDED

---

|                                |                                                     |
|--------------------------------|-----------------------------------------------------|
| To set up configuration rules: | Product Configuration Rules Designer permission set |
|--------------------------------|-----------------------------------------------------|

---

Complete these prerequisites to set up the Business Rules Engine before you create the configuration rules.

## Create a Rule Library

Rule creation requires a rule library to store and run the rules in the Standard Configurator rule engine.

1. From the App Launcher, find and select **Rule Libraries**.
2. Click **New**.
3. Enter a name and API name for the rule library.
4. Select **Configurator** as the usage type.
5. Enter the developer name of the context definition.



**Note** Use the extended SalesTransactionContext context definition that is configured for your default pricing procedure. If you don't have the extended context definition, [extend SalesTransactionContext](#) and use that.

6. Save the changes.

## Activate a Rule Library Version

Use the rule library version to store and run rules in the rule engine.

1. From the App Launcher, find and select **Rule Libraries**.
2. Click the rule library that contains the version you want to activate.
3. On the Related tab, in the rule library version related list, click the rule library version that you want to activate.
4. From the action dropdown, select **Activate**.  
Refresh the page. The rule library version is now activated.



## Enable Configuration Rules

Grant users access to set up configuration rules to validate product configurations by using the Standard Configurator.

1. From Setup, in the Quick find box, enter *Revenue Settings*, and then select **Revenue Settings**.
2. Turn on Set Up Basic Configuration Rules.

To use Standard Configurator to validate product configuration for a transaction (quote or order), the rule engine of the transaction type associated with a transaction must be StandardConfigurator. See [Specify Which Rule Engine to Use](#).

## Set Up Configurator With Constraint Rules Engine

Set up the Constraint Rules Engine to validate large and complex product configurations. Enable the Constraint Rules Engine and create the ConstraintEngineNodeStatus custom field on transaction line item objects. Map this custom field to order, quote, and asset mappings within the active context definition.

### REQUIRED EDITIONS

 **Note** Constraint Rules Engine services aren't available in Government Cloud or in orgs within the [EU Operating Zone \(OZ\)](#). For more information, contact your Salesforce account executive.

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

### USER PERMISSIONS NEEDED

---

|                                    |                                                           |
|------------------------------------|-----------------------------------------------------------|
| To enable Constraint Rules Engine: | Product Configuration Constraints Designer permission set |
|------------------------------------|-----------------------------------------------------------|

---

### [Set Up Asset Context for Asset-Based Rules and Constraints](#)

Set up an asset context definition to enable the Constraint Rules Engine to get asset data based on the selected context. Use this asset data when you configure asset-based rules and constraints in the CML Editor, and apply additional filters on the data as needed.

## Create the ConstraintEngineNodeStatus Custom Field

Before your rule designers can create advanced configuration rules and sales reps can use them, you must create the ConstraintEngineNodeStatus custom field on the Quote Line Item, Order Product (OrderItem), and Asset Action Source objects. Constraint Rules Engine uses the fields to store data for internal processing.

1. From the object management settings for Quote Line Item, go to Fields & Relationships.
2. Click **New**.
3. Select **Text Area (Long)**, and then click **Next**.
4. Enter *Constraint Engine Node Status* as the field label.
5. Enter *5000* as the length.
6. Enter *ConstraintEngineNodeStatus* as the field name.
7. Click **Next**.
8. For the profiles whose users use Constraint Rules Engine, make sure that the Read-Only option is deselected.

 **Note** For Customer Community and Partner Community users to run constraints, give these profiles also access to this field. Select the Visible option and deselect the Read-Only option for the profiles.

9. Click **Next**.
10. Deselect the page layouts, and save your changes.
11. Similarly, create the ConstraintEngineNodeStatus field on the Order Product (OrderItem) and Asset Action Source objects.

## Update SalesTransactionContext

After you create the ConstraintEngineNodeStatus field, update your extended SalesTransactionContext context definition to use the field. If you don't have the extended context definition, [extend SalesTransactionContext](#) before you add the attribute.

 **Note** Make sure to use the extended SalesTransactionContext context definition that is configured for your default pricing procedure.

1. [Edit your extended SalesTransactionContext context definition.](#)
  - a. [Add the ConstraintEngineNodeStatus attribute](#) to the SalesTrasactionItem node with INPUTOUTPUT as the type and STRING as the data type.
  - b. [Create an AssetConstraintEngineNodeStatus attribute and tag](#) in the AssetActionSource node with INPUTOUTPUT as the type and STRING as the data type.
2.  **Note** Before you create TransactionType mappings in the SalesTransactionContext context definition, in Revenue Settings, turn on Transaction processing for quotes and orders. If only the Business Rules Engine is enabled in your org, these TransactionType mappings are not required. However, if you're using the Business Rules Engine and want to enable the Constraint Rules Engine, or wish to use both, create the TransactionType mappings to switch between the rule engines at the transaction level.

In the Map Data tab, make these updates.

- a. Edit QuoteEntitiesMapping and then [map the ConstraintEngineNodeStatus attribute to the ConstraintEngineNodeStatus\\_\\_c field](#) on the QuoteLineItem object.  
In QuoteEntitiesMapping, review the context mappings and make sure that [the TransactionType attribute on the SalesTransaction node is mapped to the TransactionType field on the Quote object](#).
- b. Edit OrderEntitiesMapping and then [map the ConstraintEngineNodeStatus attribute to the](#)

[ConstraintEngineNodeStatus\\_\\_c](#) field on the OrderItem object.

In OrderEntitiesMapping, review the context mappings and make sure that [the TransactionType attribute on the SalesTransaction node is mapped to the TransactionType field on the Order object](#).

- c. Edit AssetEntitiesMapping and then [map the AssetConstraintEngineNodeStatus attribute to the ConstraintEngineNodeStatus\\_\\_c field on the AssetActionSource object](#).
  - d. Edit AssetToSalesTransactionMapping. Choose the data source as Context Definition Object, select your extended Sales Transaction context definition, and find the SalesTransactionItem object listed under it. Then, [map the AssetConstraintEngineNodeStatus attribute to the ConstraintEngineNodeStatus\\_\\_c field on that object](#).
3. After you edit the context definition, [activate it](#).
  4. Go to Object Manager, and [create a trigger for the Quote Line Item object](#) by using this apex code:

```
trigger QuoteItemTrigger on QuoteLineItem (before insert) {
    //collect QuoteActionIds
    Set<Id> quoteActionIds = new Set<Id>();

    for (QuoteLineItem qi : Trigger.new) {
        if (qi.QuoteActionId != null && qi.ConstraintEngineNodeStatus__c ==
null) {
            quoteActionIds.add(qi.QuoteActionId);
        }
    }

    if (!quoteActionIds.isEmpty()) {
        // Step 1: Get QuoteAction → SourceAsset
        Map<Id, Id> quoteActionToAssetId = new Map<Id, Id>();
        for (QuoteAction qAction : [
            SELECT Id, SourceAssetId
            FROM QuoteAction
            WHERE SourceAssetId != null
            AND Id IN :quoteActionIds
        ]) {
            quoteActionToAssetId.put(qAction.Id, qAction.SourceAssetId);
        }

        // Step 2: Get AssetActions
        List<AssetAction> assetActions = [
            SELECT Id, AssetId, ActionDate
            FROM AssetAction
            WHERE AssetId IN :quoteActionToAssetId.values()
        ];

        // Step 3: Get latest AssetAction per Asset
        Map<Id, AssetAction> assetIdToLatestAction = new Map<Id, AssetActio
n>();
```

```

        for (AssetAction aAction : assetActions) {
            AssetAction existing = assetIdToLatestAction.get(aAction.AssetId);
            if (existing == null || aAction.ActionDate > existing.ActionDate) {
                assetIdToLatestAction.put(aAction.AssetId, aAction);
            }
        }

        // Step 4: Get related AssetActionSource records
        Map<Id, Id> assetIdToActionId = new Map<Id, Id>();
        for (Id assetId : assetIdToLatestAction.keySet()) {
            assetIdToActionId.put(assetId, assetIdToLatestAction.get(assetId).Id);
        }

        List<AssetActionSource> assetActionSources = [
            SELECT ConstraintEngineNodeStatus__c, AssetAction.AssetId
            FROM AssetActionSource
            WHERE AssetActionId IN :assetIdToActionId.values() ORDER BY CreatedDate DESC
        ];

        // Step 5: Map AssetId → Status
        Map<Id, String> assetIdToStatus = new Map<Id, String>();
        for (AssetActionSource actionSource : assetActionSources) {
            if (!assetIdToStatus.containsKey(actionSource.AssetId) &&
                actionSource.ConstraintEngineNodeStatus__c != null) {
                assetIdToStatus.put(
                    actionSource.AssetId,
                    actionSource.ConstraintEngineNodeStatus__c
                );
            }
        }

        List<QuoteLineItem> toUpdate = new List<QuoteLineItem>();
        // Step 6: Set ConstraintEngineNodeStatus__c directly on Trigger.new records
        for (QuoteLineItem qi : Trigger.new) {
            if (qi.QuoteActionId != null && qi.ConstraintEngineNodeStatus__c == null) {
                Id assetId = quoteActionToAssetId != null ? quoteActionToAssetId.get(qi.QuoteActionId) : null;
                if (assetId != null && assetIdToStatus != null) {
                    String status = assetIdToStatus.get(assetId);

```

```
        if (status != null) {
            qi.ConstraintEngineNodeStatus__c = status;
        }
    }
}
}
```

## Enable Constraint Rules Engine

Enable rule designers to create advanced configuration rules and constraints, and help sales reps run the rules for transactions (quotes and orders).

- 1. From Setup, in the Quick find box, enter *Revenue Settings*, and then select **Revenue Settings**.
- 2. Turn on Set Up Configuration Rules and Constraints with Constraint Rules Engine.

 **Note** When you enable Constraint Rules Engine, AdvancedConfigurator is the default engine for transaction types in your org.

To create rules in Constraint Rules Engine, use [Constraint Builder](#).

### Set Up Asset Context for Asset-Based Rules and Constraints

Set up an asset context definition to enable the Constraint Rules Engine to get asset data based on the selected context. Use this asset data when you configure asset-based rules and constraints in the CML Editor, and apply additional filters on the data as needed.

#### REQUIRED EDITIONS

 **Note** For orgs that didn't upgrade to Summer '25, this setup isn't applicable.

Available in: Lightning Experience

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

#### USER PERMISSIONS NEEDED

|                                    |                                                           |
|------------------------------------|-----------------------------------------------------------|
| To enable Constraint Rules Engine: | Product Configuration Constraints Designer permission set |
|------------------------------------|-----------------------------------------------------------|

An asset context is required to create asset-based configuration rules and constraints. If you don't define one, the Constraint Rules Engine doesn't fetch any asset data.

## Create Custom Asset Context Definitions

To fetch the assets for a specific account, product, and asset custom field or standard field, define a custom asset context definition that meets the specified criteria.

1. [Extend the standard AssetContext\\_\\_stdctx context definition.](#)
2. [Customize the mappings](#) according to your requirements.  
You can also [edit an existing custom asset context definition.](#)

The custom asset context definitions you create are available for selection when you set up an asset context.

## Select an Asset Context for Constraint Rules Engine

Set the asset context for Constraint Rules Engine to apply asset-based configuration rules and constraints.

Make sure that you turn on the Configure Products at Runtime and the Set Up Configuration Rules and Constraints with Constraint Rules Engine settings from the Revenue Settings page before you set up the asset context.

1. From Setup, in the Quick find box, enter *Revenue Settings*, and then select **Revenue Settings**.
2. From Set Up Asset Context for Product Configurator, select the asset context definition that you want to use.

After you set the asset context, filter the asset data directly in the CML Editor based on your requirements when you define configuration rules and constraints.

## Specify Which Rules Engine to Use

Use the Transaction Type field in a quote or order to specify the rule engine to use for validating product configurations and executing configuration rules and constraints. Create Transaction Processing Type records based on your requirements, and then specify the default rule engine on the Revenue Settings page to validate product configurations and execute rules and constraints.

### REQUIRED EDITIONS



**Note** Constraint Rules Engine services aren't available in Government Cloud or in orgs within the [EU Operating Zone \(OZ\)](#). For more information, contact your Salesforce account executive.

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

---

### USER PERMISSIONS NEEDED

---

To create transaction processing type records and to select a default transaction processing type:

Customize Application

AND

View Setup and Configuration

---

#### See Also

[Connect REST API Quick Start](#)

## Create Transaction Type Records

Create Transaction Processing Type records based on the rule engine that you want to use.

Before you create a Transaction Processing Type record, make sure that [the Configure Products at Runtime setting is turned on](#).

If only Constraint Rules Engine (the Set Up Configuration Rules and Constraints with Constraint Rules Engine setting) is enabled, Product Configurator uses Constraint Rules Engine to run configuration rules.

If only Business Rules Engine (the Set Up Configuration Rules with Business Rules Engine setting) is enabled, Product Configurator uses Business Rules Engine only if StandardConfigurator is the rule engine specified in the associated Transaction Processing Type record. If the Transaction Type field on quotes and orders or the Rule Engine field on the Transaction Processing Type record is blank, Product Configurator doesn't run any configuration rules.

If both rules engines are enabled, Product Configurator uses Business Rules Engine only if StandardConfigurator is the rules engine specified in the associated Transaction Processing Type record. Else, it uses Constraint Rules Engine.

1. From Setup, in the Quick Find box, enter *Revenue Settings*, and then select **Revenue Settings**.
2. Turn on Transaction processing for quotes and orders.  
To learn how the Transaction processing for quotes and orders setting works in Transaction Management, see [Configure Transaction Management Settings](#).
3. Use TransactionProcessingType Tooling API to create a Transaction Processing Type record and specify the rule engine.  
Sales reps can't see the fields in Transaction Processing Type records. If necessary, append the name of the rule engine to the Transaction Processing Type record name.
4. If necessary, [modify the field access settings](#) of the Transaction Type field on the Quote and Order objects to provide view access to the necessary profiles.



**Example** Create transaction type records with these details.

- URI: /services/data/v63.0/tooling/subjects/TransactionProcessingType

- Request Type: POST
- Payload:

```
{ "SaveType": "Standard",  
  "Description": "<Enter a description>",  
  "DeveloperName": "<Enter a developer name>",  
  "MasterLabel": "<Enter a label>",  
  "RuleEngine": "<AdvancedConfigurator or StandardConfigurator>"}
```

 **Note** Enter the values for Description, DeveloperName, MasterLabel, and RuleEngine.

## Select the Default Rules Engine

Select a default Transaction Processing Record, containing the rule engine type, to be assigned to quotes and orders.

1. From Setup, in the Quick Find box, enter *Revenue Settings*, and then select **Revenue Settings**.
2. In the Transaction processing for quotes and orders section, select the default transaction type.

When users create quotes and orders, the Transaction Type field is automatically populated with the default Transaction Processing Type record. Admins and sales reps with the appropriate permissions can modify the Transaction Type field on quotes and orders.

When the Transaction Type field is blank and no default transaction type is defined:

- If the Set Up Configuration Rules and Constraints with Constraint Rules Engine setting is turned on, the Constraint Rules Engine runs the configuration rules.
- If the setting isn't turned on, configuration rules aren't run.

This completes the setup for Product Configurator.

To proceed with the rest of the Revenue Cloud setup, see [Set Up Revenue Cloud](#).

## Explore the Product Configurator Flow

The product configurator flow outlines the layout for presenting customizable product information in the user interface.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product

---



---

Configurator is enabled

---

Product Configurator provides an out of the box sample flow Default Product Configurator Flow. The default flow contains a screen element with the following components:

Default Product Configurator Flow

| Component Name                              | Description                                                                                                                                |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Product Configurator: Actions               | Displays a screen footer with messages from a configurator component and custom navigation buttons.                                        |
| Product Configurator: Product Attributes    | Displays product configuration attributes in a specified format.                                                                           |
| Product Configurator: Breadcrumb Navigation | Displays breadcrumb navigation for a product bundle.                                                                                       |
| Product Configurator: Header                | Displays heading information for production configuration in a specified format.                                                           |
| Product Configurator: Data Manager          | Stores product data in a hidden component and uses flow events to distribute the data to other configurator components on the flow screen. |
| Product Configurator: Messages              | Displays alert, warning, and error messages from a configurator component in the configurator footer.                                      |
| Product Configurator: Option Groups         | Displays child products of the bundled product in a specified format.                                                                      |
| Product Configurator Summary                | Displays product bundle summary, pricing, and attributes in a specified format.                                                            |
| Product Configurator: Product Header        | Displays product information fir a configurator component in a specified format.                                                           |
| Product Configurator: Transaction Header    | Displays details about a transaction and whether it's a quote, order, product list page, or product detail page.                           |



### [Clone the Default Product Configurator Flow](#)

Use the Default Product Configurator Flow as a template to design a product configurator flow that determines the layout for presenting customizable product-related information in the user interface.

### [Create a Product Configuration Flow](#)

The product configuration flow provides a layout for configuring the products and previewing the user

experience in real-time.

#### Assign Products to a Product Configuration Flow

The product configuration flow provides a simple layout for defining attribute-level details, ensuring a smooth buying experience for your customers. For products assigned to a flow, verify the product configuration experience seen during runtime.

#### Assign Product Classifications to a Product Configuration Flow

The product configuration flow provides a simple layout for defining attribute-level details, ensuring a smooth buying experience for your customers. For product classifications assigned to a flow, verify the product configuration experience seen during run time.

#### Set a Product Configuration Flow as the Default Flow

Products and product classifications without an assigned product configuration flow use the default product configuration flow to render a smooth and easy buying experience.

#### Preview the Product Configurator

See how the product configurator will look and function for your users. The product configurator can use the default product configurator flow or a customized product configurator flow. Preview the product configurator and choose the exact flow that suits the customer requirement.

## Clone the Default Product Configurator Flow

Use the Default Product Configurator Flow as a template to design a product configurator flow that determines the layout for presenting customizable product-related information in the user interface.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

### USER PERMISSIONS NEEDED

---

|                                                 |                      |
|-------------------------------------------------|----------------------|
| To clone the default product configurator flow: | Product Configurator |
|-------------------------------------------------|----------------------|

---

1. In Setup, in the Quick Find box, enter *Flow*, and then click **Flow**.
2. Select **Default Product Configurator Flow**.
3. In the Default Product Configurator Flow builder, click **Save As**.



4. Enter a label for the flow.  
The flow API name is automatically populated.
5. Save your changes, and activate the flow.

You can edit the flow to change the default configurations. To do so, edit the screen component of the cloned flow and in the flow builder chose Product Configurator: Product Attributes component. You can edit these attributes:

- Attribute Display Column: Specify the number of columns for displaying the attributes.

- **Attribute Display Style:** Specify tab or section style for how the attributes should be visually presented.
- **Attribute Categorization:** Indicate whether you want to categorize the attributes by specifying true or false.



#### See Also

[Preview the Product Configurator](#)

## Create a Product Configuration Flow

The product configuration flow provides a layout for configuring the products and previewing the user experience in real-time.

#### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

#### USER PERMISSIONS NEEDED

---

|                                         |                      |
|-----------------------------------------|----------------------|
| To create a product configuration flow: | Product Configurator |
|-----------------------------------------|----------------------|

---

1. From the App Launcher, find and select **Product Configuration Flows**.
2. Click **New**.
3. Provide the details:
  - a. For Flow Identifier, enter the API Name of the flow that you want to use as the product configuration flow.
  - b. Select a status.
 **Note** You can assign only active product configuration flows to products and product classifications.
  - c. To set this flow as the default product configuration flow for your org, select **Default**.
4. Save your changes.

#### See Also

[Clone the Default Product Configurator Flow](#)

## Assign Products to a Product Configuration Flow

The product configuration flow provides a simple layout for defining attribute-level details, ensuring a smooth buying experience for your customers. For products assigned to a flow, verify the product configuration experience seen during runtime.

## REQUIRED EDITIONS

Confirm that:

- You created a product configuration flow.
- You created a product.

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

## USER PERMISSIONS NEEDED

---

|                                                      |                      |
|------------------------------------------------------|----------------------|
| To assign a product to a product configuration flow: | Product Configurator |
|------------------------------------------------------|----------------------|

---

1. From the App Launcher, find and select **Product Configuration Flows**.
2. Click the product configuration flow that you want to assign products to.
3. On the Related tab, in the Product Configuration Flow Assignment related list, click **New**.
4. Find and select the product that you want to assign to the product configuration flow.
5. Save your changes.

Similarly, assign more products to the product configuration flow. To unassign a product from the product configuration flow, delete the product from the Product Configuration Flow Assignment related list.



**Note** You can assign only one product configuration flow to a product.

## See Also

- [Clone the Default Product Configurator Flow](#)
- [Create a Product Configuration Flow](#)

## Assign Product Classifications to a Product Configuration Flow

The product configuration flow provides a simple layout for defining attribute-level details, ensuring a smooth buying experience for your customers. For product classifications assigned to a flow, verify the product configuration experience seen during run time.

## REQUIRED EDITIONS

Confirm that:

- You created a product configuration flow.
- You created a product classification.

---

Available in: Lightning Experience

---

---

Available in: **Enterprise, Unlimited, and Developer** Editions of Revenue Cloud where Product Configurator is enabled

---



---

#### USER PERMISSIONS NEEDED

---

|                                                                     |                      |
|---------------------------------------------------------------------|----------------------|
| To assign a product classification to a product configuration flow: | Product Configurator |
|---------------------------------------------------------------------|----------------------|

---

1. From the App Launcher, find and select **Product Configuration Flows**.
2. Click the product configuration flow for which you want to assign the product classifications.
3. On the Related tab, in the Product Configuration Flow Assignment related list, click **New**.
4. Find and select the product classification that you want to assign to the product configuration flow.
5. Select the assignment type as **Configurator** to set the selected product configuration flow as the flow for the primary product or classification. Select **Dynamic Component** if you want to set it as the flow for only the dynamic components within a bundle.
6. Save your changes.  
Similarly, assign more product classifications to the product configuration flow. To unassign a product from the product configuration flow, delete the product from the Product Configuration Flow Assignment related list.



**Note** You can assign only one product configuration flow to a product classification.

#### See Also

[Clone the Default Product Configurator Flow](#)  
[Create a Product Configuration Flow](#)

## Set a Product Configuration Flow as the Default Flow

Products and product classifications without an assigned product configuration flow use the default product configuration flow to render a smooth and easy buying experience.

#### REQUIRED EDITIONS

Confirm that:

- You activated the product configurator flow.
- You created a product configuration flow.

---

Available in: Lightning Experience

---

Available in: **Enterprise, Unlimited, and Developer** Editions of Revenue Cloud where Product Configurator is enabled

---



---

#### USER PERMISSIONS NEEDED

---

|                                                    |                      |
|----------------------------------------------------|----------------------|
| To set a product configuration flow as the default | Product Configurator |
|----------------------------------------------------|----------------------|

---

---

### USER PERMISSIONS NEEDED

---

flow:

---

1. From the App Launcher, find and select **Product Configuration Flows**.
2. From the quick action menu next to the product configuration flow that you want to select as the default flow, select **Edit**.
3. Select **Default**.
4. Save your changes.

#### See Also

[Clone the Default Product Configurator Flow](#)

## Preview the Product Configurator

See how the product configurator will look and function for your users. The product configurator can use the default product configurator flow or a customized product configurator flow. Preview the product configurator and choose the exact flow that suits the customer requirement.

### REQUIRED EDITIONS

- Confirm that you assigned a product configurator flow to a product.

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---



---

### USER PERMISSIONS NEEDED

---

|                                      |                      |
|--------------------------------------|----------------------|
| To preview the product configurator: | Product Configurator |
|--------------------------------------|----------------------|

---

1. From the App Launcher, find and select **Products**.
2. Select a product that you assigned a product configurator flow to.
3. On the product details record page, from the quick action menu, select **Preview**.



## Work With the Product Configurator

Create complex product configurations with ease by using Product Configurator. Define attributes, variants, and metadata. Test and verify your product configurations with the built-in runtime experience. Your users can use the product configurator to easily configure the product options during the purchase process and view the relevant pricing.

## REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

### Launch the Product Configurator

Launch the product configurator to configure simple and bundled products and to define attributes.

### Configure Products with the Product Configurator

Products are all the items and services that you sell to customers. There are two types of products: simple products and bundled products. Simple products don't have an associated product hierarchy while bundled products generally have one. Bundled products are a group of products that are sold together as one unit.

## Launch the Product Configurator

Launch the product configurator to configure simple and bundled products and to define attributes.

## REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

## USER PERMISSIONS NEEDED

---

|                                      |                      |
|--------------------------------------|----------------------|
| To preview the product configurator: | Product Configurator |
|--------------------------------------|----------------------|

---

1. From the App Launcher, find and select **Quotes** or **Orders**.
2. If necessary, change the transaction type associated with the record.  
The Transaction Type dropdown is available only if your admin enabled the Transaction processing for quotes and orders setting. The transaction type determines [which rule engine runs product configuration rules](#).
3. From the list view, select the record that contains the line item that you want to configure.
4. On the lines tab, perform one of these actions based on your selection.
  - On the quote record details page, click **Quote Line Items**.
  - On the order record details page, click **Lines**.
5. From the quick action menu on the line item that you want to launch the Product Configurator for, select **Configure**.



The Product Configurator appears.



## Configure Products with the Product Configurator

Products are all the items and services that you sell to customers. There are two types of products: simple products and bundled products. Simple products don't have an associated product hierarchy while bundled products generally have one. Bundled products are a group of products that are sold together as one unit.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---



Simple and bundled products are further classified into static or configurable products, depending on whether the products can be configured at runtime. A bundled product consists of a parent product and child products. The child products are listed as options in the Product Configurator. The product bundle definition determines the option group that is shown during product configuration.

The Product Type field determines whether the product is a simple product or a bundled product. The ConfigureDuringSale field determines whether the product is static or configurable. The Product Type and ConfigureDuring Sale fields are available during product creation.



**Example** Hazelnut flavored instant coffee is a static simple product. However, French roast coffee grounds where users can select between medium roast or dark roast is a simple configurable product. A travel kit that contains a shampoo, conditioner, body wash, body lotion, toothbrush, and toothpaste is a static product bundle. A seller can sell a laptop with a preinstalled operating system and a free 15-month antivirus subscription. The seller can also give customers an option to purchase an extended warranty plan and wireless headphones along with the laptop. The laptop with all required and optional components is a configurable product bundle.



#### Example



**Note** When you create a configuration rule on a product bundle to take an action on a product, such as adding the product to the bundle, the configuration rule fails if the product has default attribute values that prevent the user from configuring the product.

### Configure Simple Products

Configure a simple product to update the quantity of products and view the default attribute values.

### Configure Bundled Products

Configure a bundled product to view the default options within option groups, update the quantity of products, view the default values of attributes, and select the attributes for the parent bundle and child products.

### Get the Latest Pricing



View the price updates reflect the product selections in real time.

#### Validate Product Selections Instantly

Product validation rules enhance the performance and allow you to address potential errors before saving a configuration.

#### Considerations to Save and Reuse Configurations

After you configure simple and bundled products, you can save the configurations and reuse them. You can also edit and delete saved configurations.

#### Considerations for Configuring Products with Ramp Deals

When sales reps make configuration updates to products within a time-based ramp segment, the configurations in subsequent ramp segments are also impacted.

#### Enable Non-Blocking Behavior in the Configurator

Enable non-blocking behavior so that configurator users can change multiple attribute and quantity values on a product asynchronously. The configuration page loads the user changes without waiting for the constraint engine to process each setting individually. After processing is complete, the user can save the configuration.

## Configure Simple Products

Configure a simple product to update the quantity of products and view the default attribute values.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

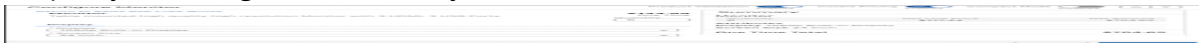
### USER PERMISSIONS NEEDED

---

|                           |                      |
|---------------------------|----------------------|
| To configure the product: | Product Configurator |
|---------------------------|----------------------|

---

1. From the App Launcher, find and select **Quotes** or **Orders**.
2. From the list view, select the record that contains the line item that you want to configure.
3. On the lines tab, perform one of these actions based on your selection.
  - On the quote record details page, click **Quote Line Items**.
  - On the order record details page, click **Lines**.
4. From the quick action menu on the line item that you want to update, select **Configure**.
5. Update the product by using one of these methods.
  - To reuse a saved configuration, click ▼ , and then select **Load Configuration**. From the list of saved configurations, select the configuration that you want to load.
  - Specify the configuration manually.



6. Save your changes.



**Example** Configure a Monitor with options to update its quantity and the display and screen size attributes.

## Configure Bundled Products

Configure a bundled product to view the default options within option groups, update the quantity of products, view the default values of attributes, and select the attributes for the parent bundle and child products.

### REQUIRED EDITIONS

Available in: Lightning Experience

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

### USER PERMISSIONS NEEDED

|                           |                      |
|---------------------------|----------------------|
| To configure the product: | Product Configurator |
|---------------------------|----------------------|

1. From the App Launcher, find and select **Quotes** or **Orders**.
2. From the list view, select the record that contains the line item that you want to configure.
3. On the lines tab, perform one of these actions based on your selection.
  - On the quote record details page, click **Quote Line Items**.
  - On the order record details page, click **Lines**.
4. From the quick action menu on the line item that you want to update, select **Configure**.
5. Update the product by using one of these methods.
  - To reuse a saved configuration, click ▼ , and then select **Load Configuration**. From the list of saved configurations, select the configuration that you want to load.
  - Specify the configuration manually.



6. Add products dynamically to the configuration.
  - a. Click **Add <Product Component Group Name>** next to the option group that you want to add products from.
  - b. Select the products, and click **Add**.



**Note** You can add the same product multiple times with a unique name attribute identifier.



**Example** Configure a Laptop Pro bundle with Laptop as the parent product and Mouse, Printer Bundle, and Warranty as the child products. Update the quantity and editable attributes of the parent and child products. For Laptop, the available attribute groups include display, graphics, processor, storage, and memory. Choose a display with the perfect size and resolution, pick a processor for your needs, select the graphics card, and find the storage and memory that match your requirements.

## Get the Latest Pricing

View the price updates reflect the product selections in real time.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

### USER PERMISSIONS NEEDED

---

|                            |                      |
|----------------------------|----------------------|
| To enable instant pricing: | Product Configurator |
|----------------------------|----------------------|

---

 **Note** Product Configurator shows amounts in the [currency unit selected in your personal settings](#) without converting the amounts.

1. From the App Launcher, find and select **Quotes** or **Orders**.
2. From the list view, select the record that contains the line item that you want to configure.
3. On the lines tab, perform one of these actions based on your selection.
  - On the quote record details page, click **Quote Line Items**.
  - On the order record details page, click **Lines**.
4. From the quick action menu on the line item that you want to enable instant pricing for, select **Configure**.
5. Turn on Instant Pricing.

For every edit or modification in the product selection, the summary component automatically shows the latest pricing. If instant pricing is turned off, the summary component isn't updated with the latest price for the selected products. To get the latest price manually, click **Update Price**.



6. Click **Save and Exit**.

## Validate Product Selections Instantly

Product validation rules enhance the performance and allow you to address potential errors before saving a configuration.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

---

**USER PERMISSIONS NEEDED**


---

|                                                                     |                                                                                                   |
|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| To enable product validation:                                       | Product Configurator permission set                                                               |
| To validate product configuration by using Constraint Rules Engine: | Product Configurator permission set AND Product Configuration Constraints Designer permission set |

---

1. From the App Launcher, find and select **Quotes** or **Orders**.
2. From the list view, select the record that contains the line item that you want to configure.
3. On the lines tab, perform one of these actions based on your selection.
  - On the quote record details page, click **Quote Line Items**.
  - On the order record details page, click **Lines**.
4. From the quick action menu on the line item that you want to enable product validation for, select **Configure**.
5. Turn on Product Validation.

For every edit or modification in the product selection, the validation rules run automatically. To manually validate the product selections, click **Validate**.



6. Click **Save and Exit**.

### Considerations to Save and Reuse Configurations

After you configure simple and bundled products, you can save the configurations and reuse them. You can also edit and delete saved configurations.

#### REQUIRED EDITIONS

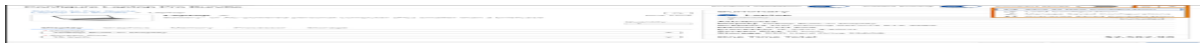
---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---



Review these considerations when you work with saved configurations.

- To update a configuration that you just saved, make changes to the configuration and click ☆. You can also update the name and description.
- To save the updates that you make to a saved configuration as a new configuration, click ▼, and then select **Save as New Configuration**.
- To save the current configuration to an existing saved configuration, select **Overwrite Saved Configuration**.

## Considerations for Configuring Products with Ramp Deals

When sales reps make configuration updates to products within a time-based ramp segment, the configurations in subsequent ramp segments are also impacted.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

Review these considerations before you configure products with line-level or group-level ramps.

- For line-level ramps, configuration stays consistent across all ramp segments.
- For group-level ramps, most configuration changes that you make in a ramp segment are applied to the current and subsequent segments. Previous segments remain unchanged to preserve data integrity.
  - Root or component quantity updates are segment-specific and don't propagate to other segments.
  - If you include or exclude a child product within a bundle in one segment, that change also propagates to all subsequent segments. If you remove a child product from a segment while it continues to be included in previous segments, you can't add it back again in subsequent segments. This is to avoid the creation of gaps in the ramp schedule.
  - Attribute changes made to a child product propagate only to the subsequent segments where that child product is already included.
- Configuration rules override your manual ramp segment configurations, and are evaluated for each segment.

For example, consider a configuration rule that states, "If T-shirt quantity is over 30, the color must be blue." Even if you manually set the color to red in a ramp segment where the T-shirt quantity is below 30, the configuration rule overrides this setting when the changes are propagated to subsequent segments. Any segment with T-shirt quantity of 31 or more automatically configures the color as blue.

## Enable Non-Blocking Behavior in the Configurator

Enable non-blocking behavior so that configurator users can change multiple attribute and quantity values on a product asynchronously. The configuration page loads the user changes without waiting for the constraint engine to process each setting individually. After processing is complete, the user can save the configuration.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

## USER PERMISSIONS NEEDED

To enable non-blocking behavior:

Product Configuration Constraints Designer permission set

When Non Blocking Enabled is turned on, if the configurator user selects values for attributes or quantity that violate rules on the constraint model, a warning message appears. For most rules, the constraint engine overrides the user selection that violates a rule, and replaces it with a valid value. For hide/disable rules, the user's selected value overrides the constraint engine, even if the user's value violates the rule.

1. Open Custom Product Configurator Flow in Flow Builder.
2. To view screen components for the flow, click the screen element on the canvas.
3. On the edit screen, select the Product Configurator Data Component.
4. In the Properties pane, click the value field for the Non Blocking Enabled property.
5. Under Screen Components, select **S01\_DataManager**, then select **isNonBlockingEnabled**.
6. In a custom flow, to turn on the Non Blocking Enabled property for all screen components, repeat steps 3-5. If you're not using a custom flow, you don't need to turn on the Non Blocking Enabled property for other screen components.
7. When you've finished setting property values, select **Done**.
8. To activate the changes, select **Save as New Version**, then select **Activate**.



**Example** Select **S01\_DataManager**, then select **isNonBlockingEnabled**, and then select **Done**.

The screenshot displays the Product Configurator Flow Builder interface. The top section shows a list of components: Product Configurator Data Manager (S01\_DataManager), Product Configurator Header (S01\_ConfigHeader), and Product Configurator Messages (S01\_Messages). The S01\_DataManager component is selected and highlighted with an orange box. To the right, the 'SCREEN COMPONENTS' pane shows a list of components, with S01\_DataManager selected and highlighted with an orange box. Below this, the 'Global Variables' pane shows a list of variables, with isNonBlockingEnabled selected and highlighted with an orange box. The bottom section shows the properties of the selected component, with the isNonBlockingEnabled property highlighted with an orange box.

## Business Rules Engine and Constraint Rules Engine

Product Configurator provides the Business Rules Engine and the Constraint Rules Engine. To work with simple product configuration rules, use the Business Rules Engine. To work with large and complex product configuration rules, use the Constraint Rules Engine. If you enable both the rules engines, you can specify which rules engine to use by default, and you can also enable sales reps to choose which

rules engine to use for each transaction.

#### REQUIRED EDITIONS

-  **Note** Constraint Rules Engine is enabled by default. If you don't need Constraint Rules Engine, disable it in Revenue Settings to prevent error messages.
-  **Note** Constraint Rules Engine services aren't available in Government Cloud or in orgs within the [EU Operating Zone \(OZ\)](#). For more information, contact your Salesforce account executive.

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

## Rules in Configurator With Business Rules Engine

When you configure products for sale, ensure that all your products have valid attributes and are compatible with other products in the offering. With configuration rules in Product Catalog Management, you can ensure that your users select the right set of products and that the product configuration aligns with the business needs and standards.

#### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

Configuration rules streamline the product configuration experience and reduce configuration errors. You eliminate fallout due to faulty product configurations when you use the validation, exclusion, and require rules. A perfect product configuration ensures accurate quotes and orders, and a smooth customer experience.

You can apply configuration rules at three levels.

- Transaction Level: The rules apply across all line items in a transaction, such as a quote or an order.
- Product Bundle Level: The rules apply across child products within a bundle.
- Product Level: The rules apply to the attributes of a single product.

Product Configurator supports different action types based on the condition criteria.

-  **Note** If you deactivate a child product, remove all references to the product in configuration rules to prevent errors.

## Action Types

| Action Type           | Description                                                                                             | Rule Scope                   | Example                                                                                                                                                                                                                                              |
|-----------------------|---------------------------------------------------------------------------------------------------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Validate              | Validate whether a product and its attributes are correctly configured by providing the user a message. | Product, Bundle, Transaction | If the user selects a laptop of brand A as part of a laptop bundle, then the docking station must be brand X, and the headphones must be brand Y. If the bundled product doesn't meet these criteria, then the user sees a validation error message. |
| Exclude               | Exclude a product along with its attributes by providing the user a message.                            | Bundle, Transaction          | The user mustn't select phone chargers of type X when they select phones of make Y in a bundle.                                                                                                                                                      |
| Require               | Include a product along with its attributes by providing the user a message.                            | Bundle, Transaction          | The user must select a specific ink cartridge when they select a specific printer as part of the printer bundle.                                                                                                                                     |
| Auto-Add              | Automatically add a product to the bundle when the rule executes.                                       | Bundle, Transaction          | If the user adds 5 laptops and is a gold status customer, then automatically add a keyboard.                                                                                                                                                         |
| Auto-Remove           | Automatically remove a product from the bundle when the rule executes.                                  | Bundle, Transaction          | If the user selects Refurbished as the type for a laptop, then automatically remove the 1-year warranty.                                                                                                                                             |
| Set Attribute         | Set one or more attributes of a product to the appropriate value.                                       | Product, Bundle              | If the user selects a laptop with advanced as configuration type, then set the value of the power adapter to 96 W.                                                                                                                                   |
| Set Quantity          | Set the quantity of a product to a certain numerical value.                                             | Product, Bundle              | If the user selects a limited edition laptop, then automatically set the quantity of the product to 1.                                                                                                                                               |
| Set Default Attribute | Set a default attribute for a product.                                                                  | Product                      | If the user selects a laptop, then select the 15-inch display by default in the display option group. Users                                                                                                                                          |



| Action Type                 | Description                                                                                                                                                                          | Rule Scope      | Example                                                                                                                           |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------|
|                             |                                                                                                                                                                                      |                 | can still choose the 13-inch or 17-inch display if they prefer.                                                                   |
| Set Default Attribute Value | Set a default value for a specific attribute of a product option.                                                                                                                    | Bundle          | If the user selects a laptop bundle, then automatically set the RAM attribute to a default value of 16GB.                         |
| Set Default Product         | Set a product option within a bundle to be automatically included when the user adds the bundle. This is useful for ensuring that a preferred product option is selected by default. | Bundle          | If the user selects a laptop pro bundle, then automatically include the Premium Support product in the bundle by default.         |
| Hide Product                | Hide the product option from the user interface when the rule executes.                                                                                                              | Bundle          | If the user selects a laptop with basic as configuration type, then hide the product 1-Year Warranty.                             |
| Hide Attribute              | Hide the attribute from the user interface when the rule executes.                                                                                                                   | Product, Bundle | If the user selects a laptop with basic as configuration type, then hide the attribute Storage.                                   |
| Hide Attribute Value        | Hide one or more attribute values from the user interface when the rule executes.                                                                                                    | Product, Bundle | If the user selects a laptop with basic as configuration type, then hide the Wireless attribute value for the Mouse attribute.    |
| Disable Product             | Disable the product option from the user interface when the rule executes.                                                                                                           | Bundle          | If the user selects a laptop with basic as configuration type, then disable the product 1-Year Warranty.                          |
| Disable Attribute Value     | Disable one or more attribute values from the user interface when the rule executes.                                                                                                 | Product, Bundle | If the user selects a laptop with basic as configuration type, then disable the Wireless attribute value for the Mouse attribute. |

When you set up configuration rules, you also define the messages that your agents or sales reps see when the rule executes. Your agents or sales reps can use the messages to rectify the product

configuration and significantly reduce the product's time to market.

### Create a Rule With Business Rules Engine

Reduce configuration errors and easily manage product validation and compatibility using configuration rules.

### Edit a Rule With Business Rules Engine

To add, update, or delete rule conditions and actions, modify the rule details or the rule criteria.

### Delete a Rule With Business Rules Engine

Delete a configuration rule when you no longer need it.

### Validate a Rule With Business Rules Engine

Rules are complex and can become outdated due to catalog changes. To maintain rule integrity, validate a configuration rule and identify the potential errors caused by catalog changes.

## Create a Rule With Business Rules Engine

Reduce configuration errors and easily manage product validation and compatibility using configuration rules.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

### USER PERMISSIONS NEEDED

---

|                                 |                                                |
|---------------------------------|------------------------------------------------|
| To create a configuration rule: | Manage Configurator with Business Rules Engine |
|---------------------------------|------------------------------------------------|

---



**Note** When you create a configuration rule on a product bundle to take an action on a product, such as adding the product to the bundle, the configuration rule fails if the product has default attribute values that prevent the user from configuring the product.

1. From the Product Catalog Management app's home page, click **Product Configuration Rules**.
2. Click **New**.
3. In the Rules Details page, specify these details:
  - a. Enter a name and description for the configuration rule.
  - b. Enter the rule duration.
  - c. Select the rule status.
  - d. To determine when this rule runs, enter the rule sequence.  
Rules with lower numbers run first when multiple rules are triggered at once.
4. Click **Next**.
5. In the Rules Criteria page, specify these details:
  - a. Select the Rule Scope.  
When the rule scope is Bundle, the configuration rule executes within the boundaries of the root

product of the bundle. When the Rule Scope is Product, the configuration rule executes for the product irrespective of where the product is in the quote or order. When the rule scope is Transaction, the configuration rule executes at the transaction level (quote or order) beyond the existing bundle or product. For example, you've created a transaction-level rule stating that Product A cannot be included with Product X in the same order or quote. Your quote includes Bundle 1 and Bundle 2. Bundle 2 contains Product X. If you try to add Product A to Bundle 1, a warning will appear. This is because Product A and Product X can't be in the same quote

b. Specify the condition to execute the rule.

The resource for each condition is either product or product classification. Each rule supports up to three conditions and eight sub-conditions. Currently, only AND logical operand is supported between conditions. The rule is executed only when all the conditions are met.



**Note** For configuration rules created before the Summer '25 release that include a condition with the NetUnitPrice sub-condition, in the Summer '25 release the NetUnitPrice sub-condition name appears as ItemDetailNetUnitPrice. The behavior stays the same.

c. Specify the action to be taken when the conditions are met.

Each rule supports up to three actions. You can add a message for each action. The type of message depends on the type of action.

6. Save your changes.

## Edit a Rule With Business Rules Engine

To add, update, or delete rule conditions and actions, modify the rule details or the rule criteria.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

### USER PERMISSIONS NEEDED

---

|                               |                                                |
|-------------------------------|------------------------------------------------|
| To edit a configuration rule: | Manage Configurator with Business Rules Engine |
|-------------------------------|------------------------------------------------|

---

1. From the Product Catalog Management app's home page, click **Product Configuration Rules**.
2. From the quick action menu next to the configuration rule that you want to edit, select **Edit**.
3. On the Rule Details page, edit the rule name, description, status, rule duration, and sequence.
4. Click **Next**.
5. On the Rule Criteria page, add, update, or delete the criteria, and the action message.
6. Save your changes.

## Delete a Rule With Business Rules Engine

Delete a configuration rule when you no longer need it.

## REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

### USER PERMISSIONS NEEDED

---

|                                 |                                                |
|---------------------------------|------------------------------------------------|
| To delete a configuration rule: | Manage Configurator with Business Rules Engine |
|---------------------------------|------------------------------------------------|

---

1. From the Product Catalog Management app's home page, click **Product Configuration Rules**.
2. From the quick action menu next to the configuration rule that you want to delete, select **Delete**.

## Validate a Rule With Business Rules Engine

Rules are complex and can become outdated due to catalog changes. To maintain rule integrity, validate a configuration rule and identify the potential errors caused by catalog changes.

## REQUIRED EDITIONS

You've created a configuration rule.

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

### USER PERMISSIONS NEEDED

---

|                     |                                      |
|---------------------|--------------------------------------|
| To validate a rule: | Product Configuration Rules Designer |
|---------------------|--------------------------------------|

---

1. From the Product Catalog Management app's home page, click **Product Configuration Rules**.
2. Click the rule that you want to validate.
3. On the configuration rule details page, click **Validate**.

After the validation is complete, identified errors are visible in the Rule Validation Error window. To fix the errors, click **Edit Rule**.



**Note** If you have an active rule on a product attribute or product bundle and modify the attribute or bundle in the product catalog, the configurator fails if the rule and the catalog are out of sync. To correct the error, deactivate the rule, then update it to be consistent with product catalog data.

## Rules and Constraints in Configurator With Constraint Rules Engine

To create configuration rules and constraints that can validate large and complex product configurations, use Constraint Rules Engine.

## REQUIRED EDITIONS



**Note** Constraint Rules Engine services aren't available in Government Cloud or in orgs within the [EU Operating Zone \(OZ\)](#). For more information, contact your Salesforce account executive.

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

### Use Constraint Builder With Constraint Rules Engine

Use Constraint Builder to create constraint models that manage complex configuration and validation for your products. Constraint models describe real-world entities and define their relationships with one another. Constraint Builder uses constraints in addition to if-then rules to customize complex products quickly and accurately.

### Convert Rules from Business Rules Engine into CML Code

Export rules from Business Rules Engine, then convert them to CML code and import them into Constraint Rules Engine.

## Use Constraint Builder With Constraint Rules Engine

Use Constraint Builder to create constraint models that manage complex configuration and validation for your products. Constraint models describe real-world entities and define their relationships with one another. Constraint Builder uses constraints in addition to if-then rules to customize complex products quickly and accurately.

## REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

To define a constraint model, you add constraints and rules. The Constraint Builder uses Constraint Modeling Language (CML), the domain-specific language of the Constraint Rules Engine, to represent the constraint model. To work in the Constraint Builder, you can choose between two interfaces:

- In the Visual Builder, use point-and-click tools to define constraints and rules, without needing to work directly with code.
- In the CML Editor, write and edit CML code to define constraints and rules. For more information on CML, see the [Constraint Modeling Language \(CML\) User Guide](#).

You can work in either of the interfaces. You can also switch between the Visual Builder and the CML Editor as you work. For example, you can define constraints and rules in the Visual Builder, and then view the code in the CML Editor to make additional changes.

 **Note** The maximum execution time for constraints is 10 seconds.

 **Note** Constraint models don't support datetime attributes.

### How User Input Order Affects Constraint Engine Behavior

The order in which a user sets attribute values for a product in PCM affects how the constraint engine computes the results.

#### Create a Constraint Model

To create a constraint model, in the Constraint Models app, name the constraint model and specify a context definition.

## How User Input Order Affects Constraint Engine Behavior

The order in which a user sets attribute values for a product in PCM affects how the constraint engine computes the results.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

For example, in this constraint for a laptop, Display and Display\_Size are attributes. The constraint specifies that if Display is 2K, Display\_Size must be 24 Inch.

```
constraint(Display == "2K Built-in Display" -> Display_Size == "24 Inch", "2K -> 24 Inch")
```

The constraint engine delivers different results depending on the order in which the user sets values for the attributes.

- If the user sets Display to 2K, the constraint engine updates the Display\_Size to 24 Inch, to validate the constraint.
- If the user sets Display\_Size to 15 Inch, then sets Display to 2K, the Display\_Size of 15 Inch violates the constraint. The constraint engine never overrides or modifies a user-selected value. Instead of updating the Display\_Size to 24 Inch, the constraint engine displays an error. To resolve the error and validate the constraint, the user can update Display to a value that is not 2K, or update Display\_Size to 24 Inch.

 **Note** For the exclude rule, the constraint engine overrides user input when necessary to validate the constraint requirements. See Exclude Rule in the [CML User Guide](#).

## Create a Constraint Model

To create a constraint model, in the Constraint Models app, name the constraint model and specify a context definition.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

### USER PERMISSIONS NEEDED

---

|                               |                                                           |
|-------------------------------|-----------------------------------------------------------|
| To create a constraint model: | Product Configuration Constraints Designer permission set |
|-------------------------------|-----------------------------------------------------------|

---

1. From the App Launcher, find and select **Constraint Models**.
2. Click **New Constraint Model**.
3. Enter a name and API name for the constraint model.
4. Select the active sales transaction context definition that the cfgStatus node is mapped to.
5. Save the constraint model.

### Define Constraints and Rules with the Visual Builder

In the Constraint Builder, use point-and-click tools in the Visual Builder to define the constraints and rules for a constraint model. Add constraints and rules to easily configure products or product bundles.

### Constraint and Rule Types in the Visual Builder

The Visual Builder provides point-and-click tools for you to easily define constraints and rules for a constraint model. Use the basic logic constraint, the conditional logic constraint, the message rule, the require rule, the exclude rule, the hide/disable rule, and the preference rule to author complex configurations for products and product bundles in your catalog.

### Use Code to Define Constraints and Rules in the CML Editor

In the Constraint Builder, use Constraint Modeling Language (CML) code to define constraints and rules for a constraint model.

### Variables, Relationships, and Associations in Constraint Models

To define how a constraint model configures products, create variables, relationships, and associations for the constraint model.

### Import Data from Salesforce Objects to Use in Constraint Models

Import data from a standard or custom Salesforce object to use in a table constraint in a constraint model. The imported data populates the columns and rows in the table constraint in CML, and saves you the step of manually entering the data.

## Define Constraints and Rules with the Visual Builder

In the Constraint Builder, use point-and-click tools in the Visual Builder to define the constraints and rules for a constraint model. Add constraints and rules to easily configure products or product bundles.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

### USER PERMISSIONS NEEDED

---

|                               |                                                           |
|-------------------------------|-----------------------------------------------------------|
| To create a constraint model: | Product Configuration Constraints Designer permission set |
|-------------------------------|-----------------------------------------------------------|

---

When you define a constraint model, keep in mind that the order in which a user sets attribute values for a product in PCM affects how the constraint engine executes the constraint model. See [How User Input Order Affects Constraint Engine Behavior](#).

1. [Create a constraint model](#).
2. In the Constraint Models app, select the constraint model you want to modify.
3. On the Details page, select the version name of the constraint model.
4. To add a new item, click **Add Item**, select **Product**, and then click **Next**.
5. Select the item or items that you want to add to the constraint model, and then save your changes.  
When you add a bundle, its product component groups and their cardinality are automatically imported from Product Catalog Management (PCM). The entire bundle structure is visible in the left panel, with child products and product classifications grouped into the respective product component groups. However, you can't define constraints or rules at the group level.
6. Under Products, select the item that you want to add a constraint or rule to.
7. In the Visual Builder window, click .
8. Select a constraint or rule type.
9. To define the constraint or rule for the selected item, add one or more expressions.
10. To create more advanced conditions, you can also group multiple expressions. To do this, define a grouping logic. Use expression numbers within parentheses '()' to form groups, and connect them with 'and', 'or', or 'xor' operators.  
For example, (1 and 2) or (3 and 4).  
If you don't define a specific grouping logic, all expressions are by default combined using an 'and' operator.
11. Add more constraints or rules as needed, and add expressions to them.
12. Save the constraint model.
13. To make the constraint model available for use, select **Activate**.

After you save a constraint model, you can switch back and forth between the Visual Builder and the CML Editor. Use the CML Editor to view and edit code for the constraint model. For more information on



CML, see the [Constraint Modeling Language \(CML\) User Guide](#).

 **Note** Some constraints and rules you create in the CML Editor can't be changed in the Visual Builder because they aren't part of the Visual Builder tools. These are labeled in the Visual Builder as not editable.

## Constraint and Rule Types in the Visual Builder

The Visual Builder provides point-and-click tools for you to easily define constraints and rules for a constraint model. Use the basic logic constraint, the conditional logic constraint, the message rule, the require rule, the exclude rule, the hide/disable rule, and the preference rule to author complex configurations for products and product bundles in your catalog.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

You add expressions to constraints and rules to define the conditions that they enforce.

Build an expression by defining these parts: a left-hand side input parameter, an operator, and a right-hand side input parameter. Your choice for each part of the expression dynamically filters the options available to you in the next part. First, select your left-hand side input parameter. Depending on the item you're building your expression for, you can choose from flexible options such as individual products, product bundles, child products within the product bundle, attributes, fields, variables defined in CML Editor, and sales transaction header. Next, select the operator. For the right-hand side input parameter, select another dynamic parameter to compare against or select the Value option to enter a static value or pick the value from a predefined list.

To create more complex conditions, you can control how multiple expressions relate to each other by grouping them. In the Grouping Logic field, define a syntax that combines your expressions. Group expressions by referencing their numbers within parentheses '(' and ')' and connecting them with the logical operators 'and', 'or', or 'xor'. For example, the logic "(1 and 2) or 3" requires that either the conditions defined in expressions 1 and 2 are both true, or that the condition in expression 3 is true.

For your grouping logic to be valid, follow these guidelines.

- Use only valid characters: Your logic can contain only expression numbers, parentheses '()', and the operators 'and', 'or', or 'xor'.
- Group expressions clearly when there are mixed operators: When the logic contains mixed operators, always group expressions by using '()' to define the exact evaluation order. For example, a grouping logic such as "1 and 2 or (3 or 4) and 5" isn't valid. To fix it, clarify the intended order of operations. Change the logic to "1 and ((2 or 3) or (4 and 5))" or "(1 and (2 or 3)) or (4 and 5)".

- Use only numbers that correspond to defined expressions: Make sure that all expression numbers that you use in the logic correspond to an expression. For example, if you have four expressions defined, you can use only the numbers 1, 2, 3, and 4.
- Reference each expression once: Every expression number can be used only once in the logic. Also, make sure that the numbers corresponding to all the defined expressions are a part of the logic.

Add a name to the constraint to make it easy to find when you work with multiple constraints in a model. Add an optional run-time message that configurator users see if their input violates the constraint conditions. Run-time messages are mandatory for the Dynamic Message Rule type.

This dynamic message rule, with the name **Optional Bundle Discount**, shows an informational message to the sales reps about an optional discount they can apply if the defined conditions are true.

The screenshot shows the 'Constraint Builder' interface for 'LaptopPrinterConstraint V1'. On the left is a 'Product Index' sidebar with a search bar and a list of products including 'LaptopProductivityBundle', 'ExternalHardDrive', 'Printer', 'Laptop', 'Office365', 'Mouse', 'LaptopStand', 'NoiseCancelingHeadphones', 'Keyboard', 'LaptopBag', and 'USBCmultiportadapthub...'. The main workspace displays a 'Dynamic Message Rule' configuration for 'Optional Bundle Discount'. The rule is titled 'Optional Bundle Discount' and has a description 'Display informational, warning, or error messages when certain conditions are true.' The 'WHEN' section contains three conditions: 1. 'Laptop > Display\_Size' equals 'Value > 15 Inch'; 2. 'Laptop > Display' equals 'Value > 1000p Built-in Display'; 3. 'Laptop > ItemNetTotalPrice' is greater than 'Value > 1000'. The 'DISPLAY' section shows the 'Message Type' as 'Informational' and the 'Run-time Message' as 'This item is eligible for a 5% discount.' The interface includes 'Sync', 'Activate', and 'Save' buttons at the top right.

When you define a constraint or rule in Visual Builder, the constraint rules engine generates the Constraint Modeling Language (CML) code for the configuration. You can view and edit the code in CML Editor, and switch between Visual Builder and CML Editor as you work. If you name the constraint in Visual Builder, the name appears as a code comment in CML Editor.



In this example, the code `message (megapackagegroup.laptop[Laptop].Display_Size == "15 Inch" && megapackagegroup.laptop[Laptop].Display == "1080p Built-in Display") || megapackagegroup.laptop[Laptop].ItemNetTotalPrice > 1000, "This item is eligible for a 5% discount.", "Info");` defines the dynamic message rule that shows an informational message to the sales reps. The code comment identifies the constraint name as Optional Bundle Discount.

### Basic Logic Constraint in the Visual Builder

Enforce a specific condition on a product, product bundle, or attribute with the basic logic constraint. Users must adhere to the defined condition to configure the associated product or product bundle successfully.

### Conditional Logic Constraint in the Visual Builder

The conditional logic constraint defines a constraint in which, if one or more conditions are true, another condition is also true. If you define a conditional constraint, users must adhere to the defined conditions in order to configure products successfully.

### Dynamic Message Rule in the Visual Builder

The dynamic message rule shows a message to users if the specified condition is true. Include a message to provide the user with information about an item, such as a promotional offer on a selected product.

### Require Rule in the Visual Builder

The require rule automatically adds a specified product to a configuration if certain conditions are true. Use the require rule to make sure that certain products are sold together.

### Exclude Rule

The exclude rule excludes an item from a quote or order when certain conditions are true. For example, use an exclude rule to specify that an order with a laptop can't include a mouse.

### Hide/Disable Rule

Hide or disable a component in a bundle, an attribute, or an attribute value when certain conditions are true, to remove the element from view, or to disable selections for it.

### Preference Rule

The preference rule applies a constraint when certain conditions are true, but allows the constrainte

engine to override the constraint if the user input violates the constraint conditions. For example, use the preference rule to specify that, when laptop display is 2K, the display size should be 15 inch or larger, but allow the configuration to continue without failing if the user selects a smaller display size, and deliver an error message indicating that display size should be 15 inch or larger.

## Basic Logic Constraint in the Visual Builder

Enforce a specific condition on a product, product bundle, or attribute with the basic logic constraint. Users must adhere to the defined condition to configure the associated product or product bundle successfully.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

In the basic logic constraint window, add an expression to define a condition for a parameter that must be true. Add more expressions as needed. Include an optional message that's shown if the user doesn't meet the required condition and prompts the user to fix the error.

## Conditional Logic Constraint in the Visual Builder

The conditional logic constraint defines a constraint in which, if one or more conditions are true, another condition is also true. If you define a conditional constraint, users must adhere to the defined conditions in order to configure products successfully.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

In the conditional logic constraint window, add one or more expressions to define a condition or conditions that must be true. Then, add an expression to define a condition that is true if the previous condition or conditions are true. For example, use a logical constraint condition to say that, when an order has six or more laptops, the shipping address must be in the United States.

Include an optional message that's shown if the user doesn't meet the required condition and prompts the user to fix the error.

## Dynamic Message Rule in the Visual Builder

The dynamic message rule shows a message to users if the specified condition is true. Include a message

to provide the user with information about an item, such as a promotional offer on a selected product.

## REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

In the dynamic message rule window, add one or more expressions to define a condition or conditions that, if true, show a message to the user. Then, enter the message in the Run-time Message field. You can also apply a message severity type to change the message's appearance and convey the level of urgency to the user.

- The informational message type has a gray banner. An informational message doesn't require the user to take any action.
- The warning type has a yellow banner. A warning message allows the user to continue working on the current task, but blocks them from taking the next step on the configuration page until they take action to address the issue described in the message.
- The error type has a red banner. An error message blocks the user from continuing with the current task on the configuration page until they fix the error described in the message.



**Note** An error message doesn't block a user working in the Transaction Line Editor (Transaction Line Table, or TLT). In that component, the user can still make changes and save the quote, even when the quote contains conditions that trigger an error message.

## Require Rule in the Visual Builder

The require rule automatically adds a specified product to a configuration if certain conditions are true. Use the require rule to make sure that certain products are sold together.

## REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

In the require rule window, add one or more expressions to define conditions. Then, select an item that must be included if those conditions are true. Specify the quantity and default attribute values for the included item as needed. For example, use the require rule to specify that a laptop must always include a blue laptop bag.



**Note** When you assign a require rule to a virtual bundle (a bundle related to the sales transaction, where the parent product has no associated price), set one Product Selling Model Option on the required product to Default. For more information on Product Selling Model Options, see [Manage](#)

## Product Selling Model in Revenue Cloud.

### Exclude Rule

The exclude rule excludes an item from a quote or order when certain conditions are true. For example, use an exclude rule to specify that an order with a laptop can't include a mouse.

#### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

Write one or more expressions to define conditions that exclude an item, then select the item that is excluded when the conditions are true.

### Hide/Disable Rule

Hide or disable a component in a bundle, an attribute, or an attribute value when certain conditions are true, to remove the element from view, or to disable selections for it.

#### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

- On a bundle, hide a component to remove it from the selection menu, or disable a component to preserve it in the menu but prevent users from selecting options for it.
- On an individual product, hide an attribute to remove it from the selection menu, or disable an attribute to preserve it in the menu but prevent users from selecting options for it.
- On an attribute, hide an attribute value to preserve it in the menu but prevent users from selecting options for it.
-  **Note** In Visual Builder, for attribute values, only the hide rule is enabled. When you apply the hide rule to an attribute value in Visual Builder, the value appears in the menu but selections are disabled. In CML code in CML Editor, for attribute values, the hide and disable rules have the same behavior.

### Preference Rule

The preference rule applies a constraint when certain conditions are true, but allows the constraint engine to override the constraint if the user input violates the constraint conditions. For example, use the

preference rule to specify that, when laptop display is 2K, the display size should be 15 inch or larger, but allow the configuration to continue without failing if the user selects a smaller display size, and deliver an error message indicating that display size should be 15 inch or larger.

#### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

Enter one or more expressions to define the conditions to be met, then enter one or more expressions to defined the preferred outcome. This outcome is overridden if the user enters input that violates the conditions to be met. Include an optional run-time message to show to the user when their input violates the rule conditions.

Use Code to Define Constraints and Rules in the CML Editor

In the Constraint Builder, use Constraint Modeling Language (CML) code to define constraints and rules for a constraint model.

#### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

#### USER PERMISSIONS NEEDED

|                               |                                                           |
|-------------------------------|-----------------------------------------------------------|
| To create a constraint model: | Product Configuration Constraints Designer permission set |
|-------------------------------|-----------------------------------------------------------|

---

In the CML Editor, you add types for a constraint model to define constraints and rules. A type corresponds mainly to a product, product bundles, or product classification. There are also special types that work differently from the main types – imported types from linked constraint models, and types corresponding to product component groups imported from Product Catalog Management (PCM). The CML Editor uses constraint modeling language (CML) to define constraint models. For more information on CML, see the [Constraint Modeling Language \(CML\) User Guide](#).

You can import data from Salesforce objects to include in CML code. See [Import Data from Salesforce Objects to Use in Constraint Models](#).

As you define a constraint model, keep in mind that the order in which a user sets attribute values for a product in PCM affects how the constraint engine executes the constraint model. For more information, see [How User Input Order Affects Constraint Engine Behavior](#).

1. [Create a constraint model](#).
2. In the Constraint Models app, select the constraint model you want to modify.
3. On the Details page, select the version name of the constraint model.
4. In the Constraint Builder header in the upper left corner, select **CML Editor**.
5. To create a new constraint model, select **New Type**.
6. Give the type a name that refers to the product, product bundle, or product classification it modifies, such as Laptop. To make the new type a child of an existing type, select a parent type.
7. Select **Create**.
8. In the left pane, select the type name.
9. To define variables, relationships, or associations for the selected type, go to the tabs for Variables, Relationships, or Associations, and follow the prompts to enter and save the required information.
10. To edit existing variables, relationships, or associations in the respective tabs, from the quick-action menu, click **Edit**, and then make the needed changes.
11. To write or edit CML code directly and define your constraints and rules, work in the CML Editor code window.
12. Save your changes.
13. To make the constraint model available for use, select **Activate**.

After you save a constraint model, you can switch back and forth between the CML Editor and the Visual Builder. Use the Visual Builder to add or edit constraints and rules with point-and-click tools.

 **Note** Some constraints and rules that you create in the CML Editor can't be edited in the Visual Builder because they're not part of the Visual Builder tools. If you try to edit one of those constraints or rules in the Visual Builder, a message prompts you to return to the CML Editor.

## Variables, Relationships, and Associations in Constraint Models

To define how a constraint model configures products, create variables, relationships, and associations for the constraint model.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

Constraint models use constraint modeling language (CML) to define variables, relationships, and associations. For more information on CML, see the [Constraint Modeling Language \(CML\) User Guide](#).

## Variables

Variables are the properties or characteristics that you define within a type. Variables can hold different kinds of data, such as strings, numbers, or lists. You can also calculate variables from other values. A variable can be a product attribute, a sales transaction item such as a product on an order, or a sales



transaction such as an order or a cart. You can also create a custom variable.

Create a standalone variable and select a data type and a value type to define the variable. For decimal or double data types, select a scale. You can also apply an automatically generated variable and select a reference library and a product to define the variable.

## Relationships

Define a relationship between types to make one type a child of another. The child inherits the constraints and rules that are applied to the parent. For example, to create a type for a laptop bundle that includes a laptop, a mouse, and a keyboard, create relationships to make the laptop, mouse, and keyboard children of the laptop bundle.

In the Import Relationships window, select a reference library and product to see the automatically generated relationships available for the product. To import relationships without the product component group structure from PCM, turn off the Import with Product Component Groups option. Select the product relationships you want to import to make them child types of the selected type.



**Note** You can reference the imported product component groups when you write your constraints. You can also write constraints on items within a product component group so that they apply only at the group level, and not at the bundle level.

## Associations

Associations are required in constraint models. Create an association to connect a type with a product record. Use associations to connect a constraint model to a product, product classification, or product component group in your product catalog. For example, create an association between a type called laptop and the laptop product in your catalog, so that the constraint model in the type applies to the product in the Configurator.

To create a type association, in the Add Type Association window, select the object category (product, product classification, or product component group), and then select the product record that you want to connect to the type. After you create the type association, the type association data table displays the ID for the type and the associated product record.

**Selected Type**  
AutoSilver

**Association Details**

\* Object Category: Product ▼

\* Object to Map:  Auto Silver X

| Associations                                     |                      |                                      |      |
|--------------------------------------------------|----------------------|--------------------------------------|------|
| <input type="text" value="Search this list..."/> |                      | <a href="#">Add Type Association</a> |      |
| Association ID                                   | Associated Object ID | CML Tag                              | Type |
| 1JExx0000004MIKGA2                               | 01txx0000006uApAAI   | AutoSilver                           | Type |

With relationship associations, you can connect a relationship of the type you're modifying to a product or product classification within a bundle in the catalog. In the Add Relationship Association window, select the type relationship (one of the types that has a relationship with the type you're modifying). Then, select a product or product classification from the bundle for the type you're modifying. After you create a relationship association, the relationship association data table displays the ID for the type relationship and the associated product records.

**Selected Type**  
AutoSilver

**Association Details**

\*Type Relationship:

Related Child Type:

\*Bundle: Auto Silver

\*Product or Product Class:

PRC Record:

| Associations                                     |                      |                                              |          |                                  |
|--------------------------------------------------|----------------------|----------------------------------------------|----------|----------------------------------|
| <input type="text" value="Search this list..."/> |                      | <a href="#">Add Relationship Association</a> |          |                                  |
| Association ID                                   | Associated Object ID | CML Tag                                      | Type     |                                  |
| 1JExx0000004MISGA2                               | 0dSxx0000000FMAEA2   | medicalpayments                              | Relation | <input type="button" value="v"/> |
| 1JExx0000004MITGA2                               | 0dSxx0000000FM9EAM   | bodilyinjurypropertydamage                   | Relation | <input type="button" value="v"/> |
| 1JExx0000004MIUGA2                               | 0dSxx0000000FMBEA2   | auto                                         | Relation | <input type="button" value="v"/> |
| 1JExx0000004MLIGA2                               | 0dSxx0000000FMBEA2   | vehicle                                      | Relation | <input type="button" value="v"/> |

## Import Data from Salesforce Objects to Use in Constraint Models

Import data from a standard or custom Salesforce object to use in a table constraint in a constraint model. The imported data populates the columns and rows in the table constraint in CML, and saves you the step of manually entering the data.

### REQUIRED EDITIONS

Available in: Lightning Experience

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

### USER PERMISSIONS NEEDED

To import object data:

Product Configuration Constraints Designer permission set

Only the first 50,000 records from the Salesforce object are imported.

1. In Object Manager, select the object that has the data you want to import, and confirm that the Constraint Rules Engine Licenseless permission set has access to the object. If the Constraint Rules Engine Licenseless permission set for the Platform Integration User isn't available in your org, contact Salesforce Customer Support.
2. In Object Manager sidebar, select Object Access.
3. In the Object Access tab, turn on the object's Read, Create, Edit, and Delete permissions for the Constraint Rules Engine Licenseless permission set.
4. To load the imported data into a constraint, in CML Editor, write an expression in the constraint model code. Use the `SalesforceTable` keyword and include the table name and column names.



**Example** In this example, data is imported to a table with four columns. The suffix `__c` indicates that the data is Salesforce custom object. `constraint(table(attr1, attr2, attr3, attr4, SalesforceTable("tablename__c", "column1name__c, column2name__c, column3name__c, column4name__c",`



**Note** If the table data is deployed when the constraint model is activated, and you add records to the table after constraint model activation, to fetch the new table data at runtime you must deactivate and reactivate the constraint model.

## Convert Rules from Business Rules Engine into CML Code

Export rules from Business Rules Engine, then convert them to CML code and import them into Constraint Rules Engine.

### REQUIRED EDITIONS

Available in: Lightning Experience

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

### USER PERMISSIONS NEEDED

|                                             |                                                           |
|---------------------------------------------|-----------------------------------------------------------|
| To import rules to Constraint Rules Engine: | Product Configuration Constraints Designer permission set |
|---------------------------------------------|-----------------------------------------------------------|

1. Go to <https://www.npmjs.com/> and install the plugins `@salesforce/plugin-data` and `@salesforce/plugin-bre-to-cml`. For detailed instructions on importing rules to Constraint Rules Engine, see information in `plugin-bre-to-cml`.
2. To export rules from Business Rules Engine and save as a JSON file, in `plugin-data`, use the `sf data export bulk` command.
3. To convert the rules from the JSON file to CML code and save it as a pair of CML and association files, in `plugin-bre-to-cml`, use the `sf cml convert prod-cfg-rules` command.
4. To import the CML and association files to the target org, in `plugin-bre-to-cml`, use the `sf cml import as-expression-set` command.

 **Example** To convert rules from JSON to CML, in plugin-bre-to-cml, use the use the `sf cml convert prod-cfg-rules` command as in this example. `$ sf cml convert prod-cfg-rules --pcr-file data/ProductConfigurationRules.json --cml-api MY_TEST --workspace-dir data --target-org breMigOrg` To import the CML and association files to the target org, in plugin-bre-to-cml, use the `sf cml import as-expression-set` command, as in this example. `$ sf cml import as-expression-set --cml-api MY_TEST --context-definition PricingTransactionCD2 --workspace-dir data --target-org tgtOrg`

The CML code for the imported rules uses a similar structure as that used to import bundle products from PCM into CML Editor. For detailed instructions on importing rules to Constraint Rules Engine, see information in plugin-bre-to-cml.

These behaviors are different in CML than in Business Rules Engine:

- In CML, conditional defaults for Set Default Attribute Value and Set Default Product aren't supported.
- In CML, auto-adding multiple instances of a product isn't supported. For example, consider this rule: if product == A and quantity > 5, auto-add B. In the Business Rules Engine, if there are 2 line items of product A with quantity greater than 5, two instances of product B are added. In CML, only 1 instance of product B is added.
- In CML, a message persists as long as a condition is true.

## Third-Party Configurator

Revenue Cloud provides extensibility options to third-party users seeking to build integrations with the revenue cloud product. Revenue Cloud supports various third-party use cases. The support includes replacing the first-party configurator with the third-party configurator, integrating third-party UI components into the first-party configurator, and creating custom configurator UI using headless configurator APIs.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

Product catalog admin uses the product configuration flows to link different configurators (first-party or third-party) to specific products and product classifications. By integrating third-party configurators, the product catalog admins can determine the appropriate configurator for each product, whether it's a first-party or a specialized third-party configurator. The product catalog admin can set the third-party configurator as the default configurator for all the products.

 **Example** Within the same organization, a Product Catalog admin can make these assignments:

- Assign the Laptop Pro Bundle product to use the **ABC** third-party configurator.
- Assign the Excavator 3000 product to use the **xyz** third-party three-dimensional configurator.
- Set all products in the Cellphone product classification to use a customized lightning flow configurator, based on the standard Revenue Cloud configurator components.
- All other products to use the first-party configurator.

### Create a Third-Party Configurator UI Component

Create a Lightning web component (LWC) to use as a custom third-party configurator UI component and add the custom component to the product configuration flow.

#### Third-Party Configurator UI Component Files

A Lightning web component that renders UI must include an HTML file, a JavaScript file, and a metadata configuration file.

#### Third-Party Configurator Lightning Flows

Third-party users can build a third-party configurator as a replacement to the first-party configurator using the screen-based lightning flows. The screen-based lightning flow is either included in the managed package or custom built in the org.

#### Third-Party Configurator UI Component Integration into First-Party Configurator

As a third-party user, apart from replacing the first-party configurator, you can also create a custom Lightning Flow UI configurator component and integrate the UI component to the first-party configurator UI. This approach provides a functional enhancement to the existing first-party configurator UI.

#### Custom Configurator Development with First-Party Configurator APIs

As a third-party user, build a custom configurator UI by using the public Configurator APIs. Incorporate the existing first-party configurator UI components for a familiar look and feel, and deliver a personalized experience by using the configurator API functionalities.

#### Use Third-Party Configurator for Specific Products or Classifications

Configure product or classifications by using the third-party configurator instead of the default configurator.

#### Use Place Quote API or Place Order API for Data Transfer

Transfer data from a custom configurator to a quote or order in Revenue Cloud by using the Place Quote API or Place Order API.

#### Data Types for Configurator User Interface

The Apex classes contain data accessible through the user interface. The Configurator's Data Manager acts as the primary user interface component that exports data to other components. As a third-party user you can pass the data into your own user interface component without the need for separate queries to fetch data.

## Create a Third-Party Configurator UI Component

Create a Lightning web component (LWC) to use as a custom third-party configurator UI component and add the custom component to the product configuration flow.

## REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---



---

USER PERMISSIONS NEEDED

---

|                                                           |                      |
|-----------------------------------------------------------|----------------------|
| To add a custom component to a product configuration flow | Product Configurator |
|-----------------------------------------------------------|----------------------|

---

1. Use Salesforce Developer Experience (Salesforce DX) to create an LWC to use as a custom UI component. See [Create Lightning Web Components](#).
2. To add the custom component to the product configuration flow and define the logic for sending and receiving data, in the custom component folder add the required HTML, JavaScript, and js-meta.xml files.
3. To enable the custom component to communicate with the Data Manager component in the product configuration flow, trigger Lightning Message Service (LMS) events. Make sure that the custom component doesn't directly call the configurator API or any Save APIs. See [Communicate Across the DOM with Lightning Message Service](#).
4. In Flow Builder, drag the custom component from the sidebar to the Product Configurator Flow layout. See [Flow Builder Tour](#).
5. In the Flow Builder sidebar, open the Data Manager component to view the available outputs, and then link the appropriate data manager outputs to the custom component.

## See Also

[Third-Party Configurator UI Component HTML File](#)

[Third-Party Configurator UI Component JavaScript File](#)

[Third-Party Configurator UI Component Metadata Configuration File](#)

## Third-Party Configurator UI Component Files

A Lightning web component that renders UI must include an HTML file, a JavaScript file, and a metadata configuration file.

## REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Developer**, **Enterprise**, and **Unlimited** Editions of Revenue Cloud where Product Configurator is enabled

---

[Third-Party Configurator UI Component HTML File](#)

Every UI Lightning web component (LWC) has an HTML file with the root tag `<template>`. The HTML file follows the naming convention `componentName.html`, for example, `myComponent.html`.

### Third-Party Configurator UI Component JavaScript File

Every component has a JavaScript file. For the third-party configurator UI component, this file contains all the logic for sending and receiving data and interacting with the configurator.

### Third-Party Configurator UI Component Metadata Configuration File

The configuration file defines the metadata values for the component, including supported targets and the design configuration.

## Third-Party Configurator UI Component HTML File

Every UI Lightning web component (LWC) has an HTML file with the root tag `<template>`. The HTML file follows the naming convention `componentName.html`, for example, `myComponent.html`.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

This example shows the basic structure for an HTML file for a component.

```
<template>
  <p>My third party configurator flow component</p>
</template>
```

### See Also

[Create a Third-Party Configurator UI Component](#)

## Third-Party Configurator UI Component JavaScript File

Every component has a JavaScript file. For the third-party configurator UI component, this file contains all the logic for sending and receiving data and interacting with the configurator.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

This JavaScript example uses Lightning Message Service events to send and receive data.

```
import {api, LightningElement} from 'lwc';
```

```

import {MessageContext, publish} from 'lightning/messageService';

import CONFIGR_CHANNEL from "@salesforce/messageChannel/lightning__productConfigurator_notification";

const LMS_EVENTS = Object.freeze({
  VALUE_CHANGE: "valueChanged",
  NAVIGATE: "navigate",
  CLOSE_PREVIEW: "closePreview",
  TOGGLE_INSTANT_PRICING: "toggleInstantPricing",
  TOGGLE_RULES_VALIDATION: "toggleRulesValidation",
  TOGGLE_COMPACT_LAYOUT: "toggleCompactLayout",
  UPDATE_PRICES: "updatePrices",
  VALIDATE_PRODUCT: "validateProduct",
  CLONE_ITEMS: "cloneItems",
});

export const STATE_FIELDS = Object.freeze({
  IS_SELECTED: "isSelected",
  QUANTITY: "Quantity",
  ATTRIBUTE_FIELD: "AttributeField",
  PRODUCT_SELLING_MODEL: "ProductSellingModel",
  PRICING_TERM_UNIT: "PricingTermUnit",
  SUBSCRIPTION_TERM: "SubscriptionTerm",
  SELLING_MODEL_TYPE: "SellingModelType",
  PRICE_BOOK_ENTRY: "PricebookEntry",
  IS_DELETED: "Deleted",
  UNIT_PRICE: "UnitPrice",
  CUSTOM_PRODUCT_NAME: "CustomProductName"
});

export default class MyComponent extends LightningElement {

  @api transactionId;
  @api transactionLineId;
  @api currentTransactionLineId;
  @api parentName;
  @api origin;
  @api messages;
  @api header;
  @api optionGroups;
  @api summary;
  @api navigationRoute;
  @api searchInfo;

```



```

@api currencyCode;
@api transactionRecord;
@api headerTitle;
@api isDesignTime;
// ... do @api annotations for each of the input properties you want from
// Data Manager

subscription = null; // Stores the subscription to the message service

// Lifecycle hook that subscribes to the message channel when the componen
t is initialized
connectedCallback() {
  this.subscribeToMessageChannel();
}

// Subscribes to the Lightning Message Service channel to receive message
subscribeToMessageChannel() {
  if (!this.subscription) {
    this.subscription = subscribe(
      this.messageContext,
      CONFIGR_CHANNEL,
      (message) => this.handleMessage(message)
    );
  }
}

handleMessage(message) {}

// Send the message to LMS
sendMessageToDataManager() {
  const bulkMessagePayload = {
    action: LMS_EVENTS.VALUE_CHANGE,
    data: [
      // Change 1: update the root product's quantity field
      {
        key: ["0QLxx0000004jGGGAY"],
        field: STATE_FIELDS.QUANTITY,
        value: 100
      },
      // Change 2: update the root product's attribute
      {
        key: ["0QLxx0000004jGGGAY"],
        field: STATE_FIELDS.ATTRIBUTE_FIELD,
        attributeId: "0tjxx0000000001AAA",

```

```

        value: "New Value"
    },
    // Change 3: update another one of root product's attributes
    {
        key: ["0QLxx0000004jGGGAY"],
        field: STATE_FIELDS.ATTRIBUTE_FIELD,
        attributeId: "0tjxx0000000001AAB",
        value: "0xxxx0000000001AAB"
    },
    // Change 4: create a picklist attribute that does not exist
(non-defaulted picklist)
    {
        key: ["0QLxx0000004jGGGAY"],
        field: STATE_FIELDS.ATTRIBUTE_FIELD,
        attributeId: "0tjxx0000000001AAC",
        value: "0xxxx0000000001AAB"
    },
    // Change 5: select a static child product
    {
        key: ["0QLxx0000004jGGGAY"],
        productRelatedComponentId: '0dSxx00000000XtEAI', // the st
atic child PRC to select
        field: STATE_FIELDS.IS_SELECTED,
        value: true
    },
    // Change 6: select a dynamic option with a child option below
it
    {
        field: STATE_FIELDS.IS_SELECTED,
        selectedDynamicOptions: dynamicOptionsForInputPayload,
        productRelatedComponent: prcForInputPayload
    },
    // Change 7: change the PSM for the root product from onetime
to termed
    {
        key: ["0QLxx0000004jGGGAY"],
        field: STATE_FIELDS.PRODUCT_SELLING_MODEL,
        value: {
            psmId: "0jPxx00000002JZEAX",
            pbeId: "01uxx000000A0tKAAX"
        }
    },
    // Change 8: deselect a child static option
    {

```

```

        key: ["0QLxx0000004jGGGAY", '0QLxx0000004jGHGAY'],
        field: STATE_FIELDS.IS_SELECTED,
        value: false
      }
    ]
  };

  publish(MessageContext, CONFIGR_CHANNEL, bulkMessagePayload);
}
}

```

### Lightning Message Service for Third-Party Configurator UI Components

Use Lightning Message Service (LMS) events to enable a third-party component to communicate with the Data Manager component in the product configurator flow. Add LMS events to the JavaScript file for the component to send and receive data.

#### See Also

[Create a Third-Party Configurator UI Component](#)

### Lightning Message Service for Third-Party Configurator UI Components

Use Lightning Message Service (LMS) events to enable a third-party component to communicate with the Data Manager component in the product configurator flow. Add LMS events to the JavaScript file for the component to send and receive data.

#### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

This example shows available LMS events.

```

export const LMS_EVENTS = Object.freeze({
  VALUE_CHANGE: "valueChanged",
  NAVIGATE: "navigate",
  CLOSE_PREVIEW: "closePreview",
  TOGGLE_INSTANT_PRICING: "toggleInstantPricing",
  TOGGLE_RULES_VALIDATION: "toggleRulesValidation",
  TOGGLE_COMPACT_LAYOUT: "toggleCompactLayout",
  UPDATE_PRICES: "updatePrices",
  VALIDATE_PRODUCT: "validateProduct",
});

```

```
CLONE_ITEMS: "cloneItems",
});
```

This example shows modifiable fields on sales transaction items.

```
export const STATE_FIELDS = Object.freeze({
  IS_SELECTED: "isSelected",
  QUANTITY: "Quantity",
  ATTRIBUTE_FIELD: "AttributeField",
  PRODUCT_SELLING_MODEL: "ProductSellingModel",
  PRICING_TERM_UNIT: "PricingTermUnit",
  SUBSCRIPTION_TERM: "SubscriptionTerm",
  SELLING_MODEL_TYPE: "SellingModelType",
  PRICE_BOOK_ENTRY: "PricebookEntry",
  IS_DELETED: "Deleted",
  UNIT_PRICE: "UnitPrice",
  CUSTOM_PRODUCT_NAME: "CustomProductName"
});
```

This example shows the payload for the `valueChanged` event:

```
const bulkMessagePayload = {
  action: LMS_EVENTS.VALUE_CHANGE,
  data: [
    // Change 1: update the root product's quantity field
    {
      key: ["0QLxx0000004jGGGAY"],
      field: STATE_FIELDS.QUANTITY,
      value: 100
    },
    // Change 2: update the root product's attribute
    {
      key: ["0QLxx0000004jGGGAY"],
      field: STATE_FIELDS.ATTRIBUTE_FIELD,
      attributeId: "0tjxx0000000001AAA",
      value: "New Value"
    },
    // Change 3: update another one of root product's attributes
    {
      key: ["0QLxx0000004jGGGAY"],
      field: STATE_FIELDS.ATTRIBUTE_FIELD,
      attributeId: "0tjxx0000000001AAB",
      value: "0xxxx0000000001AAB"
    }
  ]
};
```

```

    },
    // Change 4: create a picklist attribute that does not exist (non-defaulted picklist)
    {
        key: ["0QLxx0000004jGGGAY"],
        field: STATE_FIELDS.ATTRIBUTE_FIELD,
        attributeId: "0tjxx0000000001AAC",
        value: "0xxxx0000000001AAB"
    },
    // Change 5: select a static child product
    {
        key: ["0QLxx0000004jGGGAY"],
        productRelatedComponentId: '0dSxx00000000XtEAI', // the static child PRC to select
        field: STATE_FIELDS.IS_SELECTED,
        value: true
    },
    // Change 6: select a dynamic option with a child option below it
    {
        field: STATE_FIELDS.IS_SELECTED,
        selectedDynamicOptions: dynamicOptionsForInputPayload,
        productRelatedComponent: prcForInputPayload
    },
    // Change 7: change the PSM for the root product from onetime to term
    {
        key: ["0QLxx0000004jGGGAY"],
        field: STATE_FIELDS.PRODUCT_SELLING_MODEL,
        value: {
            psmId: "0jPxx00000002JZEAX",
            pbeId: "0luxx000000A0tKAAX"
        }
    },
    // Change 8: deselect a child static option
    {
        key: ["0QLxx0000004jGGGAY", '0QLxx0000004jGHGAY'],
        field: STATE_FIELDS.IS_SELECTED,
        value: false
    }
]
};

publish(MessageContext, MessageChannel, bulkMessagePayload);

```

## Third-Party Configurator UI Component Metadata Configuration File

The configuration file defines the metadata values for the component, including supported targets and the design configuration.

### REQUIRED EDITIONS

The configuration file follows the naming convention `componentName.js-meta.xml`, such as `myComponent.js-meta.xml`.

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

Set up the `js-meta.xml` file to expose the component to Lightning Flow and add it to the Product Configurator flow template.

This example includes the input properties you can import from the Data Manager component or other variables on the default product configurator flow. If you create a custom configurator flow with other properties, you can add those to your `js-meta.xml` file.

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
  ~ Copyright 2024 salesforce.com, inc.
  ~ All Rights Reserved
  ~ Company Confidential
-->

<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>60.0</apiVersion>
  <isExposed>true</isExposed>
  <masterLabel>My Third Party Configurator Component</masterLabel>
  <description>My custom component that I will use in the
    Configurator UI</description>
  <targets>
    <target>lightning__FlowScreen</target>
  </targets>
  <targetConfigs>
    <targetConfig targets="lightning__FlowScreen">
      <!-- INPUT PROPERTIES -->
      <property
        name="transactionLineId"
        type="String"
        required="true"
```

```

        label="Line Item ID"
        description="Enter the ID of the transaction line to confi
gure."
        role="inputOnly" />
    <property
        name="currentTransactionLineId"
        type="String"
        required="true"
        label="Current Line Item ID"
        description="The ID of the transaction line to launch the
configurator."
        role="inputOnly" />
    <property
        name="transactionId"
        type="String"
        required="true"
        label="Transaction ID"
        description="Enter the ID of the transaction to configur
e."
        role="inputOnly" />
    <property
        name="addedNodes"
        type="apex://ProductConfig.SalesTransactionItem[]"
        label="Added Nodes"
        description="Enter the API name of the flow resource that
contains a collection of Apex ProductConfig__SalesTransactionItem records asso
ciated with the parent record transaction."
        role="inputOnly" />
    <property
        name="parentName"
        type="String"
        required="true"
        label="Parent Name"
        description="Enter the name or title of the parent transac
tion that the configuration pertains to."
        role="inputOnly" />
    <property
        name="origin"
        type="String"
        required="true"
        label="Object Name"
        description="Enter the API name of the transaction recor
d's parent object."
        role="inputOnly" />

```

```

    <property
      name="messages"
      type="apex://ProductConfig.Message[]"
      label="Messages"
      description="A collection of Apex ProductConfig__Message records that contain alerts, warnings, and errors encountered by the component."
      role="inputOnly" />
    <property
      name="optionGroups"
      type="apex://ProductConfig.OptionGroup[]"
      label="Option Groups"
      description="A collection of ProductConfig__OptionGroup records that contain product option data grouped by product category."
      role="inputOnly" />
    <property
      name="summary"
      type="apex://ProductConfig.PricingSummary"
      label="Transaction Summary"
      description="An Apex ProductConfig__PricingSummary record that contains summary information for the specified transaction ID, including all price-related details."
      role="inputOnly" />
    <property
      name="attributeCategories"
      type="apex://ProductConfig.AttributeCategory[]"
      label="Attribute Categories"
      description="A collection of Apex ProductConfig__AttributeCategory records that contain details about the attributes groups that are grouped by attribute category."
      role="inputOnly" />
    <property
      name="header"
      type="apex://ProductConfig.Product"
      label="Product Details"
      description="An Apex ProductConfig__Product record that contains details about the root-level product, including product name, description, and pricing."
      role="inputOnly" />
    <property
      name="navigationRoute"
      type="apex://ProductConfig.NavigationInfo[]"
      label="Navigation Information"
      description="A collection of Apex ProductConfig__Navigation

```



```

nInfo records that contain breadcrumb navigation information for the UI."
    role="inputOnly" />
    <property
        name="searchInfo"
        type="apex://ProductConfig.SearchItemInfo[]"
        label="Search Info"
        description="A collection of Apex ProductConfig__SearchItemInfo records that contain option information for the search bar."
        role="inputOnly" />
    <property
        name="currencyCode"
        type="String"
        label="Currency Code"
        description="The currency code used for the transaction."
        role="inputOnly" />
    <property
        name="transactionRecord"
        type="apex://ProductConfig.TransactionRecord"
        label="Transaction Record"
        description="An Apex ProductConfig__TransactionRecord record that contains transaction record details, including transaction origin, parent name, and ID."
        role="inputOnly"/>
    <property
        name="isClassContext"
        type="Boolean"
        label="Product Classification Preview Mode"
        description="A Boolean value that indicates whether the record being previewed was a product classification record."
        role="inputOnly" />
    <property
        name="isDesignTime"
        type="Boolean"
        label="Preview Mode"
        description="A Boolean value that indicates whether a product or product classification record was viewed in preview mode."
        role="inputOnly" />
    <property
        name="headerTitle"
        type="String"
        label="Configurator Title"
        description="The title that was used for the configurator screen heading."
        role="inputOnly" />

```

```

    <property
      name="isInstantPricingToggleEnabled"
      type="Boolean"
      label="Show Instant Pricing Toggle"
      description="A Boolean value that indicates whether the in
stant pricing toggle was shown."
      role="inputOnly" />
    <property
      name="isCompactLayoutEnabled"
      type="Boolean"
      label="Compact Mode"
      description="A boolean value that indicates whether compac
t mode is enabled."
      role="inputOnly" />
    <property
      name="isInstantPricingEnabled"
      type="Boolean"
      label="Instant Pricing"
      description="A Boolean value that indicates whether the in
stant pricing toggle was enabled. If false, the Update Prices button is shown
for manual price refresh."
      role="inputOnly" />
    <property
      name="isProductValidationEnabled"
      type="Boolean"
      label="Product Validation"
      description="A Boolean value that indicates whether produc
t validation was enabled."
      role="inputOnly" />
    <property
      name="showPrices"
      type="Boolean"
      label="Price Visibility"
      description="A Boolean value that indicates whether pricin
g information was shown at runtime."
      role="inputOnly" />
    <property
      name="isConfiguratorDisabled"
      type="Boolean"
      label="Disabled"
      description="A Boolean value that indicates whether this c
omponent was disabled."
      role="inputOnly" />
    <property

```

```

        name="isApiInProgress"
        type="Boolean"
        label="API Call Status"
        description="A Boolean value that indicates whether the AP
I call is in progress."
        role="inputOnly" />
    <property
        name="isPriceRampEnabled"
        type="Boolean"
        label="Ramp Deals Enabled"
        description="A Boolean value that indicates whether Ramp D
eals are enabled for this product."
        role="inputOnly" />
    <property
        name="tabs"
        type="apex://ProductConfig.NavigationInfo[]"
        label="Navigation Tabs"
        description="A collection of Apex ProductConfig__Navigatio
nInfo records to display tabs in the Configurator."
        role="inputOnly" />
    <property
        name="salesTransactionItems"
        type="apex://ProductConfig.SalesTransactionItem[]"
        label="Sales Transaction Items"
        description="A collection of Apex ProductConfig__SalesTran
sactionItem records that contain sales transaction item details."
        role="inputOnly" />
    <property
        name="searchResultOptionId"
        type="String"
        label="Search Result Option Id"
        description="The ID of the option selected from the search
bar."
        role="inputOnly" />
</targetConfig>
</targetConfigs>
</LightningComponentBundle>

```

### See Also

[Create a Third-Party Configurator UI Component](#)

## Third-Party Configurator Lightning Flows

Third-party users can build a third-party configurator as a replacement to the first-party configurator

using the screen-based lightning flows. The screen-based lightning flow is either included in the managed package or custom built in the org.

## REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

Both first-party and third-party configurators receive the same data when the screen-based Lightning Flow launches during a configuration process.

```
"configuratorContext": {
  "transactionId": <the quote/order ID being configured>,
  "transactionLineId": <the quote/order item ID being configured>,
  "parentName": <the quote/order name>,
  "origin": <"Quote" | "Order">,
  "addedNodes": [{
    "id": <a synthetic quote/order item Id>,
    "pricebookEntry": <Pricebook Entry Id>,
    "productSellingModel": <Product Selling Model Id>,
    "unitPrice": <Unit Price>,
    "quantity": <Quantity>,
    "product": <Product2 Id>,
    "businessObjectType": <QuoteLineItem or OrderItem>
  }]
}
```

When the screen-based Lightning Flow built with the required data format, the screen-based Lightning Flow can be assigned to specific products, product classifications, or even entire organizations, where it functions as the designated configurator.

Third-party configurators offer greater flexibility. Third-party configurators can have their own solutions from scratch, bypassing features like Salesforce Pricing, Context Service, Configurator API, and Business Rules Engine. Third-party configurators can use any Salesforce APIs that are required. There's no obligation to use any of the Revenue Cloud specific APIs.

### See Also

[Set a Product Configuration Flow as the Default Flow](#)

## Third-Party Configurator UI Component Integration into First-Party Configurator

As a third-party user, apart from replacing the first-party configurator, you can also create a custom

Lightning Flow UI configurator component and integrate the UI component to the first-party configurator UI. This approach provides a functional enhancement to the existing first-party configurator UI.

#### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

To add a widget, include the Lightning Web Component (LWC) in the ISV managed package. After the LWC component is available, the product catalog admin modifies the default first-party configurator to add the third-party user's custom LWC components. The widgets can interact with the first-party configurator data, including:

- Product details and bundle configurations.
- Product pricing and pricing results.
- Product attributes and their specified values.



**Example** Consider a map widget that a third-party user develops by using the LWC platform's map UI component. When you select a location on the map, the widget automatically updates the latitude and longitude coordinates stored as product attributes. Sales reps can use this widget to configure the location for a specific product.

## Custom Configurator Development with First-Party Configurator APIs

As a third-party user, build a custom configurator UI by using the public Configurator APIs. Incorporate the existing first-party configurator UI components for a familiar look and feel, and deliver a personalized experience by using the configurator API functionalities.

#### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

The configurator API includes Revenue Cloud services such as Context Service, Business Rules Engine, Constraint Rules Engine, and Salesforce Pricing. Third-party users can enable or disable the Revenue Cloud features. The configurator API is available as a Connect API.

Apart from the first-party configurator APIs, third-party users can also build a custom configurator by using the elements of the first-party configurator UI.

## Use Third-Party Configurator for Specific Products or Classifications

Configure product or classifications by using the third-party configurator instead of the default configurator.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

Specify the use of a third-party configurator by assigning a Product Configuration Flow linked to the third-party configurator to products or classifications. As a result, the third-party configurator is launched when you configure these products.

### See Also

[Assign Products to a Product Configuration Flow](#)

[Assign Product Classifications to a Product Configuration Flow](#)

## Use Place Quote API or Place Order API for Data Transfer

Transfer data from a custom configurator to a quote or order in Revenue Cloud by using the Place Quote API or Place Order API.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise**, **Unlimited**, and **Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

The APIs offer flexibility to either include or exclude the first-party configurator logic. Further, third-party users can use the first-party configurator API tasks such as validating the bundle structure, applying Salesforce Pricing rules, or implementing qualification rules.



### Example

```
"configurationInput": ["skip" / "runAndAllowErrors" / "runAndBlockErrors"], // Default runAndBlockErrors
"configurationOptions": {
  "validateProductCatalog": true,
  "validateAmendRenewCancel": true,
```

```

"executeConfigurationRules": true,
"addDefaultConfiguration": true
}
// rest of Place Quote API payload
}

```

## Data Types for Configurator User Interface

The Apex classes contain data accessible through the user interface. The Configurator's Data Manager acts as the primary user interface component that exports data to other components. As a third-party user you can pass the data into your own user interface component without the need for separate queries to fetch data.

### REQUIRED EDITIONS

---

Available in: Lightning Experience

---

Available in: **Enterprise, Unlimited, and Developer** Editions of Revenue Cloud where Product Configurator is enabled

---

The effective method to interact with the user interface is to import data through the Flow user interface properties that the Data Manager exports.

The effective way to modify quantities, attribute values, product options, and other state changes is to trigger LMS events for any updates your user interface component makes.

The Data Manager simplifies data management in your user interface component. It constantly updates and sends the latest information whenever changes occur. This creates a seamless flow where you don't need to actively listen for updates. Instead, you can simply import or watch the data directly from the flow properties.

The Configurator User Interface Data Manager exports the following Apex data types. This data is readily available for use by third-party configurators in the user interface, eliminating the need for them to query the data separately.

### Apex Data Type: ProductConfig\_\_Product

ProductConfig\_\_Product

| Field Name | Type   |
|------------|--------|
| id         | String |
| quantity   | Double |

| Field Name                 | Type                            |
|----------------------------|---------------------------------|
| isConfigurable             | Boolean                         |
| isActive                   | Boolean                         |
| quantityReadOnly           | Boolean                         |
| isAssetizable              | Boolean                         |
| isQuantityEditable         | Boolean                         |
| isSelected                 | Boolean                         |
| isPersisted                | Boolean                         |
| isHidden                   | Boolean                         |
| disabled                   | Boolean                         |
| isSoldOnlyWithOtherProds   | Boolean                         |
| name                       | String                          |
| nodeType                   | String                          |
| productCode                | String                          |
| productType                | String                          |
| productComponentGroups     | List<OptionGroup>               |
| attributeCategories        | List<AttributeCategory>         |
| productKey                 | List<String>                    |
| additionalFields           | Map<String, Object>             |
| prices                     | List<Price>                     |
| productClassification      | ProductClassification           |
| productSellingModelOptions | List<ProductSellingModelOption> |
| qualificationContext       | QualificationContext            |
| minOptions                 | Integer                         |
| maxOptions                 | Integer                         |
| sequence                   | Integer                         |



## Apex Data Type: ProductConfig\_\_Attribute

| ProductConfig__Attribute |              |
|--------------------------|--------------|
| Field Name               | Type         |
| id                       | String       |
| name                     | String       |
| label                    | String       |
| attributeCategoryId      | String       |
| attributeNameOverride    | String       |
| code                     | String       |
| dataType                 | String       |
| defaultValue             | String       |
| description              | String       |
| productKey               | List<String> |
| isCloneable              | Boolean      |
| isConfigurable           | Boolean      |
| isHidden                 | Boolean      |
| isPriceImpacting         | Boolean      |
| isReadOnly               | Boolean      |
| isRequired               | Boolean      |
| assetAttributeValue      | String       |
| attributeValue           | String       |
| attributeKey             | String       |
| attributePicklistValue   | String       |
| businessObjectType       | String       |

## Apex Data Type: ProductConfig\_\_AttributeCategory

| ProductConfig__AttributeCategory |        |
|----------------------------------|--------|
| Field Name                       | Type   |
| code                             | String |

| Field Name | Type            |
|------------|-----------------|
| id         | String          |
| name       | String          |
| attributes | List<Attribute> |

## Apex Data Type: ProductConfig\_\_ConfiguratorContext

ProductConfig\_\_ConfiguratorContext

| Field Name        | Type                       |
|-------------------|----------------------------|
| transactionLineId | String                     |
| parentName        | String                     |
| origin            | String                     |
| transactionId     | String                     |
| addedNodes        | List<SalesTransactionItem> |

## Apex Data Type: ProductConfig\_\_Message

ProductConfig\_\_Message

| Field Name | Type   |
|------------|--------|
| text       | String |
| type       | String |

## Apex Data Type: ProductConfig\_\_NavigationInfo

ProductConfig\_\_NavigationInfo

| Field Name | Type         |
|------------|--------------|
| index      | Integer      |
| name       | String       |
| productKey | List<String> |